$\sqrt{Eurocomp}$ LGP-21 Subroutine Handbook

Equinox Deschanel

June 2026

"Our need will be the real creator."

— Plato, *The Republic*

"The enemy of art is the absence of limitations."

— Orson Welles

Contents

1 Notes on the Translation **4**
- 1.1 Number representation . . . 4
- 1.2 Document organization . . . 6
- 1.3 LGP-21 coding conventions . . . 7
  - 1.3.1 Address notation . . . 7
  - 1.3.2 Calling sequences . . . 7
- 1.4 Repeated/redundant information . . . 9
- 1.5 Acknowledgements and disclaimers . . . 10

2 Trigonometric functions **11**
- 2.1 Sine-Cosine 1 . . . 11
- 2.2 Sine-Cosine 2 . . . 13
- 2.3 Arctangent 1 . . . 15
- 2.4 Arctangent of a Vector 1 . . . 18
- 2.5 arcsin/arccos 1 . . . 20

3 Logarithmic Functions **22**
- 3.1 Exponential Function 1 . . . 22
- 3.2 Log 1 . . . 24

4 Root Extraction **28**
- 4.1 Square Root 1 . . . 28
- 4.2 n-th Root . . . 30

5 Floating Point **32**
- 5.1 Floating Point Interpreter 1 . . . 32

6 Floating-Point Interpretive Systems **32**
- 6.1 Floating-Point Interpretive System 1 . . . 32
- 6.2 Floating Point Interpreter 2 . . . 51

7 Number Conversions **60**
- 7.1 Decimal/Hexidecimal Conversion . . . 60

8 Program input **61**
- 8.1 Program Loader 1 . . . 61
  - 8.1.1 THE EFFECT OF THE PROGRAM JUMP SWITCHES 68
- 8.2 Program Loader 2 . . . 72
  - 8.2.1 PURPOSE . . . 72

9 Data Input **82**
- 9.1 Data Input 1 . . . 82
- 9.2 Data Input 2 . . . 82
- 9.3 Data Input 3 . . . 82
- 9.4 Data Input 4 . . . 82

9.5 Data Input 5 82

10 Data Output **83**
10.1 Data Output 1 83
10.2 Data Output 2 83
10.3 Data Output 3 83
10.4 Data Output 4 83
10.5 Data Output 5 83
10.6 Data Output 6 83

11 Hexadecimal Output **84**
11.1 Hexadecimal Output 2 84
11.2 Hexadecimal Output 3 84
11.3 Hexadecimal Output 4 84

12 Alphanumeric Output **85**
12.1 Alphanumeric Output 1 85
12.2 Alphanumeric Output 2 85

13 Hexadecimal Input **86**
13.1 Hexadecimal Input 1 86

14 Specialized Input/Output **87**
14.1 Angle or Direction I/O 87

15 Tracing **88**
15.1 Trace 1 88

16 Memory Dumps **89**
16.1 Memory Dump 1 89
16.2 Memory Dump 2 89

17 Sorting **90**
17.1 Sort 1 90
17.2 Sort 2 90
17.3 Sort 3 90
17.4 Sort 4 90

18 Conversion to Binary **91**
18.1 Binary Conversion of Integers 1 91

19 Backup Utilities **92**
19.1 Backup 1 92
19.2 Tape Duplicator 1 92

20 Binary Searching **93**
20.1 Table Search 1 (Binary Search) 93

21 Programming Utilities **94**
21.1 Loop Driver Utility 94

22 Device Support **95**
22.1 High-speed Punch Output 95

23 Bibliography **96**

1 Notes on the Translation

1.1 Number representation

Introduction to fractional fixed-point architecture

The LGP-21's number representation is very different from 21st century computing platforms. It is not at all unique; many early computers used it, including the IBM 701[3], the ILLAC I[4], the Elliot 405[5], the RPC-4000[10], and the LGP-21's predecessor, the LGP-30[6]. This architecture is known as the IAC architecture, and was first proposed by John von Neumann for EDSAC[1].

A fixed-point fractional representation treats all numbers as fractions lying between -1 and $+1$. As von Neumann pointed out[2], the multiplication of 2 n-bit integers results in a value that may require $2n$ bits to represent it, which could easily overflow a register, but the multiplication of *fractions* always results in a number no larger, and usually smaller, than both multiplicands (e.g., $0.5 \times 0.5 = 0.25$). The answer still requires $2n$ bits to represent, but the extra bits end up in the *least* significant digits, preventing overflow from occurring.

The design uses a combined register pair to contain the $2n$ bits; on the LGP-21, the accumulator receives the most significant half, and a hidden register not available to the programmer gets the least significant half. To reduce complexity, the LGP-21 simply throws this second half away. Since these are the least significant digits, the (unscaled) error introduced is 2^{-31} or less.

This advantage in multiplication is offset by a problem this creates with division: fractional multiplication *shrinks* numbers, but fractional division (potentially) *expands* them.

In a fractional fixed-point architecture, dividing two numbers requires that the absolute magnitude of the divisor has to be *strictly greater than* the absolute magnitude of the dividend.

$$|\text{Divisor}| > |\text{Dividend}|$$

Getting this wrong means that the theoretical quotient is ≥ 1.0, and such a value cannot physically fit in the accumulator. The architecture maintains the *hardware* binary point just after the sign bit, so a value ≥ 1.0 results in an overflow.

Von Neumann wrote that "*[t]he programmer must see to it* [emphasis mine - ED] that the left-shift [scale factor] is sufficiently large to ensure this inequality."burks1946preliminary[2] The cognitive load of remembering where the *implied* binary point is (via the scaling factor) vs. the *physical* binary point (implemented by the hardware) is placed solely on the programmer.

Modern embedded systems use similar notation (referred to as *Q-notation* or *fixed-point scaling*) to define this manual placement of the implicit binary point within a computer word, and in fact, the original version of this document specifically refers to the scaling factor as q.

LGP-21 Fractional Fixed-point Implementation

The LGP-21 treats all 31-bit words strictly as fractions ranging from -1 to $+1$, with the physical binary point fixed immediately to the right of the (left-most) sign bit. To work with integers or mixed numbers, the programmer must mentally shift this binary point to the right by n bits (as indicated by the @n notation).

For example, a value at texttt@3 allocates 3 bits for the integer portion of a number (allowing values up to 7.999999) and the remaining 27 bits for the fraction. As noted above, the programmer is *entirely responsible* for tracking this scale factor across operations, and must manually shift, multiply, or divide intermediate values as needed to maintain the desired scale factor.

Addition/Subtraction Pitfalls

Addition and subtraction require that the scale factors (`@n`) of both operands be identical before the operation is executed. The LGP-21 treats every 31-bit word as a raw fixed-point fraction. If you add two numbers with different q factors without shifting them appropriately, the machine will blindly align their bits and perform the operation, smashing incommensurate units of completely different mathematical weights into the same columns.

For example, if you attempt to add 1.0 scaled at `@1` to 1.0 scaled at `@4` without normalizing them first:

	1.0 at `@1` →	`0 100000...`	(Hardware sees 2^{29})
+	1.0 at `@4` →	`0 000100...`	(Hardware sees 2^{26})
	Raw Sum →	`0 100100...`	(Hardware sees $2^{29} + 2^{26}$)

Depending on which scale factor you *think* the result is in, your answer is now either 1.125 (if you assume the result is `@1`) or 9.0 (if you assume `@4`). The machine will not throw an error, trigger an overflow, or warn you; it will simply calculate perfectly represented garbage because the binary point in the two numbers is in *two different places*.

It is vital to understand that the LGP-21 hardware itself does not know about scale factors and has no way to check them or track them. Scale factor changes occur during multiplication and division: multiplication *adds* the scale factors, and division *subtracts* them. After a multiplication or division, the result must be shifted left or right to ensure that the result conforms to the scale factor desired.

As an example, if an `@1` value were to be divided by an `@9` one, the resulting scale factor would be -8, meaning that to return it to an `@1` value, the result would need to be shifted right *8* bits, and for an `@9` result, it would need to be shifted right *17*(!) bits.

In the subroutines documented below, I've carried over the original `@n` notation where it was used, most often for referring to values in storage (e.g., θ@9) or the accumulator (accumulator`@1`).

1.2 Document organization

First, the code described in this document spans a wide range of functions. Some of it really is subroutines, some of it what we'd call functions (subroutines that return a specific value or result), and some is what we'd call a program, utility. or monitor. The original document simply calls everything a suroutine, and what the document calls a "calling sequence" is how the code is executed. This means that some "calling sequences" really are a set of machine instructions that perform a subroutine call, while others are a series of operations that the machine operator must perform to load and start the program.

Because the actual function of the code is highly variable, I've chosen to refer to the individual programs as appropriate, whether a function, subroutine, utility, etc. "Calling sequence" is altered as appropriate for the kind of program in question; it may be termed "Execution" or "Usage" instead.

The document is organized in broad section, with the broadly broken into mathematical functions, a pair of floating-point calculators, input and output support code, utilities, and a grab-bag of miscellaneous other algorithms and utilities: sorts, searches, data maintenance/backup, and programming tools. (The tool that automates self-modifying code loops is particularly engaging!)

Code names

The programs do not have names! This is probably because the name was pretty arbitrary as far as the machine was concerned – you can't load a program by name, etc.; it's all simply reference material for humans. As such the names are assigned to organize the programs, from broad functional groups down to exactly which program is which.

The first character of the name establishes which broad area a program falls into: `B` is mathematical, `D` is matrixes, `J` is input/output, and so on. The second character refines this: `B1` is trig functions; `B3` is exponential/log functions; `B4` is roots. There is no rule as to how the refinement is assigned other than personal taste.

The numeric part of the name refines the classification further. `B1` is trig; `B1-10` is sine and cosine, `B1-11` is arctangent, `B1-12` is arcsin/arccos. Within that numeric classification, we have a point number that differentiates implementations. So `B1-10.0` (we number the point classifications from `0`) is mathematics (`B`), trigonometry (`1`), sine/cosine (`-10`), first implementation (`.1`). As a sample, `B1-10.0` is the *first* sine and cosine implementation, which accepts degrees; `B1-10.1` is the *second* sine and cosine implementation, which accepts radians.

These break up reasonably well into sections and subsections, so I've done that to make it easier to navigate the table of contents. I have not reordered any of the programs to ensure that any cross-referencing between the German and English versions of the document were easy to do.

1.3 LGP-21 coding conventions

1.3.1 Address notation

Subroutine addresses in the original text are written as `L` or, for offsets into program code, `L+n`.

Addresses in the main program are written as α or $\alpha + n$.

The LGP-21 is too primitive to have anything like a linkage editor, so it's the programmer/operator's responsibility to use the LGP-21's Program Input Routine[1] to load subroutine code and automatically relocate it as it's loaded. The usage of `L` for the base address of code is to remind the reader that they will need to relocate these library functions by hand and use the new base addresses in their code to make subroutine calls work properly.

1.3.2 Calling sequences

Because the LGP-21 libraries were written by multiple programmers, and there was no oversight body to enforce a One True Way of doing things, there is a Cambrian explosion of alternatives. Very capable programmers not only coded their algorithms, but cleverly spaced their storage so that the data was under the disk's read head when an instruction wanted to access memory.[2]

Fortunately, none of those programming tricks are used here[3], but the "let a million flowers bloom" philosophy does show itself in the multiple ways there are to call subroutines.

Accumulator-only parameters

One calling convention streamlines the call as much as possible by loading the parameter into the accumulator, setting the return address in a `U` instruction in the subroutine at an agreed-upon offset, and then jumping to the subroutine:

Location	**Instruction**	**Address**	**Description**
$\alpha - 1$	`B`	`N`	Bring N into the accumulator.
α	`R`	`L+`*`offset`*	Store the return address in the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.

In this example a `B` loads the argument into the accumulator, but any instruction that gets the appropriate value into the accumulator is just as valid. The `R` saves the return address (α+2) in the subroutine, and the main program's `U` to the start of the subroutine calls it. Execution of the main program resumes just after the `U` to `L` at $\alpha + 2$.

[1] or Rhys Weatherly's `lgp21-tools` assembler, which can include and relocate subroutines.

[2] See the story of Mel[?], who originally wrote for the LGP-30; his code was famous for using instructions as data, and exploiting the weird corners of the instruction set to do things like have loops with no exit which nevertheless eventually exit.

[3] As far as I know. Heaven knows what's actually lurking in the library code itself.

This means that calling a subroutine implies that code *must* be modified for a return to work: specifically, a `U` (unconditional jump) instruction in the subroutine must be altered to set the return address. Multiple calls to a subroutine therefore will clobber any previous return address.

This means that re-entrant code is not natively supported by the hardware. Programmers must ensure that they carefully manage the hierarchy of subroutine calls to prevent a second call to a subroutine, directly or indirectly, before it returns.

Inlined parameters

A second calling convention, with longer, in-memory parameter lists, looks like this:

Location	Instruction	Address	Description
$\alpha - 1$	`E`	`0000`	Exit from the Interpretive System.
α	`R`	`L`	Modify subroutine instruction with key word address.
$\alpha + 1$	`U`	`L`	Unconditional branch to subroutine entry.
$\alpha + 2$	…	…	A fixed number of parameter words
$\alpha + n$	—	—	Return to main program.

This call looks exceedingly strange if you're used to the previous calling convention. If we were doing exactly the same operation as before (altering a `U` instruction to set the return, then jumping to the subroutine), we'd just jump right back, so that can't be right, and it isn't.

This convention uses the `R` instruction to load the address of the *parameters* and store it into a `B` instruction at the start of the subroutine instead. Jumping to the subroutine loads the address of the parameters ($\alpha + 2$) into the accumulator, allowing the subroutine to add an offset to this address to set it to $\alpha + n$, which can then be stored locally in a `U` instruction to do the actual return to the caller, and to have a pointer to the parameters as well!

Deep Wizardry: Floating-Point Parameters

A third, even more astounding calling sequence passes a multi-word floating-point[4] number to a subroutine as an argument.

Location	Instruction	Address	Description
$\alpha - 1$	`B`	`N`	Bring word 1 of floating-point number N.
α	`R`	`L+offset`	Store the return address ($\alpha + 2$) inside the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.

[4]While the LGP-21 lacks floating-point *hardware*, it relies entirely on a software-driven floating-point math *interpreter*. See Section **??**.

Now wait a minute, I hear you say. A floating-point number is *three* words long. That cannot possibly fit in the single-word hardware accumulator—what kind of trickery is this?

You're right. The full floating-pint number doesn't get loaded; only its first word does. But the `B` instruction itself at cell $\alpha - 1$ now contains the exact piece of metadata the subroutine needs: *the starting memory address `N` of the floating-point block*[5].

This explains why the manual repeatedly states that "loading this value into the accumulator by any means" is acceptable for some subroutines, but does *not* say that for this one[6]. The `B` here is essential to the program working. It relies on the code modified by the `R` instruction to find and read its caller:

1. The subroutine picks up the return address ($\alpha + 2$) deposited by the `R` instruction into cell `L+offset`.
2. It subtracts 3 from that value to locate cell $\alpha - 1$ (the `B` instruction).
3. It modifies the next instruction, a `B`, to set its address to $\alpha - 1$.
4. The modified `B` instruction loads the literal 32-bit instruction word stored at $\alpha - 1$...that contains the address of the floating-point number.
5. It strips away the `B` opcode bits using a bitmask, leaving behind only the raw address `N`.
6. Now the subroutine has the address of the first of the three words making up the floating-point number, and can proceed.

The value loaded into the accumulator at step $\alpha - 1$ is *entirely ignored*. The `B` instruction's sole purpose in this calling sequence is to serve as an *executable static data pointer*, which the subroutine finds and dissects to find words 2 and 3 (`N+1` and `N+2`) of the floating-point parameter.

1.4 Repeated/redundant information

Because this is a reference manual meant for use by people who were casually programmers and principally something else entirely, this manual fairly consistently repeats things like the standard `B/R/U` instruction sequence used to call a function as noted in **??**. I have tried to condense this where possible.

Some subroutines use unusual parameterization (such as patching the values of words in the subroutine itself!), or are actually programs to be loaded and run, and I have retained all of the setup information and/or programming information as it was originally written in these cases.

[5] My reaction was "You are *kidding* me" with a significant expletive titre.

[6] Gun, see foot.

1.5 Acknowledgements and disclaimers

Thanks in particular to David Lovett of Usagi Electric for resurrecting a real, physical LGP-21 and starting me down this road; Rhys Weatherly for the lgp21-tools, and Paul Kimpel for the wonderful LGP-21 web emulator that has given me such a lot of pleasure learning the machine.

Thanks also to the many people whose names I don't know[7], who have tirelessly made sure that important historical documents like the original German version of this manual and the programs that go with it have been preserved. This document is my small contribution to this historical preservation effort, and I am immensely grateful to those giants upon whose shoulders I stand.

This work *is* a translation, and I cannot guarantee I have been completely accurate, whether from misunderstanding or transcription error. I've done my best to be sure my interpretation of what's documented here is correct, but I have not run or tested all of the programs referenced, and I can't make any guarantees as to their fitness and accuracy, nor can I be sure that the document I've translated here is in fact 100 percent accurate itself! I have made corrections where errors were obvious, but I may have missed things.

To that end, this document will be up on GitHub; please feel free to submit PRs for corrections and errata there if you find anything that needs a fix. I intend to CC license this document such that it can be freely built upon and distributed, but that credit for the basic work is retained.

[7] Send me patches via GitHub so I can credit you, please!

2 Trigonometric functions

2.1 Sine-Cosine 1

B1-10.0

PURPOSE

`B1-10.0` calculates the sine or cosine of an angle value in degrees. If the angle supplied is greater than 360°, the value is reduced to an angle between 0° and 360° before the function value is calculated. A 9th-degree polynomial is used for approximation.

`B1-10.0` uses the standard LGP-21 calling sequence. The angle argument must supplied in the accumulator@9. The function value, including its sign, will be in the accumulator@1 when the subroutine returns.

Memory usage

Memory for program	1 track
Buffer usage	Sectors 2, 4, 5, 6, 7, 45 of track 63

Input

The input variable for B1-10.0 is an angle in degrees, θ, for which the function value is to be calculated. θ must be in the accumulator@9 when the jump to the subroutine occurs. The angle is reduced to an angle between 0° and 360° by the subroutine before evaluation.

Calling Sequence

Different calling sequences are provided for sine and cosine. Only one value (either sine or cosine) can be calculated per subroutine call.

Sine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@9 into the accumulator
α	`R`	$L + 49$	; Store return address at end of sub
$\alpha + 1$	`U`	L	; call the sine routine

This is a standard function call, with the argument passed in the accumulator and the result returned in it.

Cosine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@9 into the accumulator
α	`R`	`L+49`	; Store return address at end of sub
$\alpha + 1$	`U`	`L+4`	; call the cosine routine

The calling sequence for cosine is exactly the same as for sine, except the subroutine is entered at the origin address of the `B1-10.0` subroutine plus 4 words.

Output

When the subroutine returns, the properly-signed function value is in the accumulator@1.

OPERATION

- **Accuracy:** The value is accurate to within 5 x 10^{-7}.
- **Time required:** approximately 775 ms.
- **Program input:** the tape contains the program in two versions:
  - Relocatable, hexadecimal. Valid for J1-10.0 (section 8.1).
  - Relocatable, decimal, in coding sheet format.
- **Required I/O devices:** none.
- **Overflow:** ignored on entry and not reset on exit.

2.2 Sine-Cosine 2

B1-10.1

PURPOSE

`B1-10.1` calculates the sine or cosine of an angle θsupplied in radians. The absolute value of the argument must not exceed 7.9999999. Before calculating the function, the subroutine reduces the angle θ to an angle β whose absolute value in radians is $\leq$1.75. A 14th-degree Taylor series is used to calculate the function.

The program `B1-10.1` is a standard LGP-21 subroutine. The angle argument must be in the accumulator when the subroutine is called. The function value is in the accumulator@1 when the function returns.

Memory usage

Memory for program	1 track
Buffer usage	Sectors 8 and 49 of track 63

Input

The input variable for B1-10.1 is an angle θ, whose function value is to be calculated. θ must be in the accumulator@3 and have a value <= 7.9999999 when the call to the subroutine occurs. The subroutine reduces the angle to an equivalent angle β with an absolute value <= 1.75.

Calling Sequence

Sine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@3 into the accumulator
α	`R`	`L+16`	; Store return address at end of sub
$\alpha + 1$	`U`	`L`	; call the sine routine

This is a standard LGP-21 function call; the argument is loaded into the accumulator before the call, and the result is returned in the accumulator.

Cosine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@3 into the accumulator
α	`R`	`L+16`	; Store return address at end of sub
$\alpha + 1$	`U`	`L+45`	; call the cosine routine

The calling sequence is exactly the same as for the sine, except that we jump to the cosine entry point to the subroutine, 45 words past the start of the subroutine.

Output

On return, the function value with the appropriate sign is in the accumulator@1.

OPERATION

- **Accuracy:** The error in radian measure is less than 8 x 10^{-9}.
- **Time required:** approx. 1200 ms.
- **Program format:** the tape contains the program in two ways:
  - Relocatable, hexadecimal, valid for J1-10.0 (section 8.1).
  - Relocatable, decimal, in coding sheet format.
- **Required input/output devices:** None.
- **Overflow:** Ignored and not reset.

2.3 Arctangent 1

B1-11.0

PURPOSE

`B1-11.0` calculates the arctangent of any number less than 512. The returned value is signed and expressed in degrees. A 15th-degree polynomial is used to approximate the function.

`B1-11.0` is a standard LGP-21 subroutine. The argument must be in the accumulator; the arctangent value is in the accumulator on return to the main program. [Translator note: see section below on result scaling.]

This is a standard LGP-21 function call: the argument is loaded into the accumulator before the call, and the result is returned in the accumulator.

Memory usage

Memory for program	1 track
Buffer usage	Sectors 4, 5, 6, 7, 8, 9, 10, 13, 50, and 51 of track 63

Input

The input variable for `B1-11.0` is the value N@9 whose arctangent is to be computed. This value must be loaded into the accumulator before the subroutine is called.

Calling sequence

Location	Instruction	Address	Description
$\alpha - 1$	`B`	N	; Load N@9 into the accumulator
α	`R`	`L+51`	; Store return address at end of sub
$\alpha + 1$	`U`	`L`	; call the arctangent routine

This is a standard function call. Load the argument into the accumulator before the call; the value is returned in the accumulator.

Output

Upon returning to the main program, the signed principal value of the arctangent in degrees is in the accumulator@9. The absolute value will be between 0° and 89.90°.

OPERATION

- **Accuracy:** The absolute error is no greater than 5 x 10^{-7} degrees.
- **Time required:** 950 ms.

- **Program format -** the tape contains the program in two ways:
  - Arbitrarily relocatable hexadecimal, suitable for J1-10.0 (section 8.1).
  - arbitrarily relocatable decimal, in coding sheet format.
- **Required input/output devices:** None.
- **Overflow:** Neither considered at input nor reset at output.

REMARKS

A slight modification to the subroutine call can be made to allow larger input values whose arctangents are closer to 90° (absolute) by changing the contents of memory locations $\alpha + 56$ and $\alpha + 59$ to rescaled values of 1 and 2.

Memory location	Contents (old)	Contents (new)
`L+56`	1@9	1@q
`L+59`	2@9	2@q

The argument passed in the accumulator must be rescaled as well to the same q scaling value when jumping to the subroutine.

Translator's notes

Arctangent calculations near 90 degrees

The q value above is an LGP-21 fixed-point scaling factor[8], allowing the routine to process much larger numbers and yield accurate results for angles close to 90°. Unlike modern mathematical functions, which seamlessly handle scaling or accept floating-point infinities, or which permit passing more than one parameter to a subroutine(!), this function requires the programmer to adjust the input boundary parameters by patching the constants inside the function itself.

In the default configuration (@9), the maximum value the machine can physically represent is $2^9 - 1$ or 511.999999. As an angle approaches 90°, its tangent approaches infinity. Since the arctangent is the inverse function, this means that your application cannot use the @9 constants if the input values need to be larger than 511; calculating with the @9 constants will overflow.

Changing the scaling of the patched values (and of the incoming parameter!) to a larger scaling factor q, of 10 or greater, expands the range of the input. We still cannot represent mathematical infinity, but we can process much larger input values, allowing us to get significantly closer to a true 90° output.

[8] see 1.1

Warnings

The subroutine does not validate that the q value used for the patchable constants and your input argument match! If you patch the internal constants at $\alpha + 56$ and $\alpha + 59$ to use a custom scale factor (say, @12) , you must scale your input argument to that exact same scale factor before calling the routine. Cross-scaled parameters and constants will silently yield erroneous results and possible overflow conditions.

Because the word size is a fixed 31 bits, increasing q to accommodate larger integers directly reduces the number of bits available to the fractional side of the word. If you push q too high, you will lose precision on return values for smaller input values.

2.4 Arctangent of a Vector 1

B1-11.1

PURPOSE

The program `B1-11.1` calculates θ such that, given x and y, the following holds:

$$y = \sqrt{x^2 + y^2} \sin\theta \tag{1}$$

$$x = \sqrt{x^2 + y^2} \cos\theta \tag{2}$$

The x and y values may be positive or negative; the function operates correctly in all quadrants of the cartesian plane. The values x and y must be given at the same q. The result represents a partial result that can be converted into degrees or degrees of arc.

`B1-11.1` uses a standard LGP-21 subroutine call, but the y value must be stored in the subroutine's designated parameter storage, and x must be in the accumulator prior to the call. Upon returning to the main program, the result will be in the accumulator@1.

Memory usage

Memory required for program	2 tracks, 17 sectors
Buffer usage	None

Input

The input variables for the program are the values x and y , which must have the same q. y must be stored in the subroutine, and x must be in the accumulator when the subroutine is called.

Calling Sequence

Location	**Instruction**	**Address**	**Description**
$\alpha - 3$	`B`	Y	; Load Y@q into the accumulator
$\alpha + 2$	`C`	`L+34`	; save into subroutine parameter storage
$\alpha + 1$	`B`	`X`	; load X@q into the accumulator
α	`R`	`L+10`	; set return address
$\alpha + 1$	`U`	`L`	; call the arctangent routine

Store y in the designated parameter storage, `L+34` (any operation that puts it there is fine). Load x into the accumulator (again, any operation accomplishing this is fine). Remember that x and y must be scaled to the same q value.

Output

On return from the subroutine, the accumulator@1 will contain the result. To convert to degrees, multiply this value by 180; to convert to radians, multiply the value by π.

Operation

- **Accuracy:** No error exceeds +4 x 10^{-8} in the partial result.
- **Time required:** approx. 1160 ms.
- **Program format -** the tape contains the program in two versions:
  - Relocatable hexadecimal, valid for J1-10.0 (section 8.1).
  - Relocatable decimal, in coding sheet format.
- No input/output devices are required.
- **Overflow:** Ignored at subroutine call, and not reset at exit.

2.5 arcsin/arccos 1

B1-12.0

PURPOSE

The program `B1-12.0` calculates either the arcsine or the arccosine of a given number from -1 $\leq N \leq$ 1. The signed return value is expressed in degrees. A 7th-degree polynomial is used for approximation.

`B1-12.0` is a standard LGP-21 subroutine. When jumping to the subroutine, the argument must be in the accumulator, and the return address must be stored in the subroutine. Upon returning to the main program, the desired function value is in the accumulator@9.

Memory usage

Program storage	2 track, 32 sectors
Buffer usage	Sectors 12, 16, 18, 19, 20, 21, 23, 24, 28 of Track 63

Input

The value N@1 , with -1 $\leq N \leq$ 1 must be in the accumulator when the function is called.

Calling Sequence

Arcsin

Location	Instruction	Address	Description
$\alpha - 1$	`B`	N	; Load N@1 into the accumulator
α	`R`	`L+21`	; set return address
$\alpha + 1$	`U`	`L`	; call the arcsin routine

Load N into the accumulator, set the return address at `L+21` , and jump to `L` to compute the arcsin. Mainline execution continues after the jump.

Arccos

Location	Instruction	Address	Description
$\alpha - 1$	`B`	N	; Load N@1 into the accumulator
α	`R`	`L+21`	; set return address
$\alpha + 1$	`U`	`L+211`	; call the arcsin routine

Load N into the accumulator, set the return address at `L+21` , and jump to `L+211` to compute the arccos. Mainline execution continues after the jump.

Output

Output is returned in the accumulator@9 in radians; the value of the function is - $\pi/2$ to $\pi/2$ for arcsin, and 0 to π for arccos.

Notes

Since calculating the arcsine or arccosine requires calculating a square root, a program for this has been included in this subroutine. It can be used independently with the following calling sequence:

```
α − 1   B   value    ; load value with an even q
                     ; into the accumulator
                     ; (see translator notes)
α       R   L+150    ; set return address
α + 1   U   L+100    ; call the square root routine
```

The square root is returned in the accumulator. Note that the q value of the result is *half* of the argument's q value.

Translator's note

There's a lot hiding inside that throwaway "the q value of the result is half of the argument's q value". You may also be wondering why the q value of the argument *must* be even, especially since we're doing all this math in @1's and @9's.

This all goes back, again, to the fractional fixed-point math of the LGP-21. In modern floating-point math, the hardware (or software) manages all the exponents for you, but on the LGP-21, something interesting happens when we take the square root of a scaled number.

If you have a value X scaled to factor q, its internal integer representation in the machine is actually $X = x \cdot 2^q$. When you take the square root of that internal machine representation, look at what happens to the exponent:

$$\sqrt{X} = \sqrt{x \cdot 2^q} = \sqrt{x} \cdot 2^{q/2} \tag{3}$$

If your scale factor q is an odd number (like the standard @1 or the @9 used elsewhere in these library functions), then $q/2$ is a fraction (4.5)! You cannot shift a binary word by 4.5 bits, meaning that you can't make the result of the square root operation commensurate to any q.

Forcing the programmer to provide an input with an even q (like @2, @8, or @18) means that the resulting scale factor after the square root function finishes will be an integer (@1, @4, or @9, respectively) and that the result will be valid.

3 Logarithmic Functions

3.1 Exponential Function 1

B3-10.0

PURPOSE

The program `B3-10.0` calculates the exponential function K^x, where K = 2, e, or 10 and $x : -1 \leq x \leq 1$. `B3-10.0` expects x in the accumulator, and returns K in the accumulator.

Memory usage

Program storage	63 sectors
Buffer usage	None

Input

The input value x must be scaled to $-1 \leq x \leq 1$ and have a q @1.

Calling Sequence

Each of the exponential functions has a separate entry point into the subroutine:

2^x

```
α − 1   B   N      ; load N@1 into the accumulator
α       R   L+9    ; set return address
α + 1   U   L      ; call the 2^x routine
```

e^x

```
α − 1   B   N      ; load N@1 into the accumulator
α       R   L+9    ; set return address
α + 1   U   L+2    ; call the ex routine
```

10^x

```
α − 1   B   N      ; load N@1 into the accumulator
α       R   L+9    ; set return address
α + 1   U   L+3    ; call the 10^x routine
```

Each of the entry points is a standard LGP-21 subroutine call; mainline execution resumes directly following the unconditional jump to the subroutine.

Output

On return, the function value is in the accumulator@4.

Notes

- **Accuracy:** No error is greater than 5 x 10^{-8}.
- **Time required:** approx. 775 ms.
- **Program format -** the tape contains the program in two versions:
  - Relocatable hexadecimal, valid for J1-10.0 (section 8.1)
  - Relocatable decimal, in coding sheet format.
- **Required input/output devices:** None.
- **Overflow:** The program neither checks overflow upon input nor resets it.

Translator notes

While the input to these functions is strictly restricted to @1, the routine returns the final calculated value in the accumulator scaled to @4. This choice of scale factor represents a best-case compromise with the LGP-21's fixed-point architecture.

Because the maximum possible input value to any of these functions is 1.0, the maximum possible outputs are predetermined:

$$\begin{aligned} 2^1 &= 2.0 \\ e^1 &= 2.718281828\ldots \\ 10^1 &= 10.0 \end{aligned}$$

Recalling that the representation of the largest possible value for any of these functions as an integer is 10, or 1010_2 — four bits in binary — this means we need to have four bits in front of the binary point, but not more, to represent the maximum value of 10.0: a q of @4.

If we instead chose to return the data at a standard library scale like @9, this would throw away bits on the right side of the binary point to provide space to on the left-hand side of the binary point to represent values much larger that we would ever need. If, on the other hand, we chose to use @1, this would limit the return value's maximum to 1.99, causing an overflow on 2^1 and most positive e^x and 10^x calculations.

By standardizing the output to @4, `B3-10.0` allows the maximum possible value (10.0) for any of these functions to be returned while preserving an optimal 26 bits of precision for the fractional portion of the result for smaller numbers.

3.2 Log 1

B3-11.0

PURPOSE

The program `B3-11.0` calculates the logarithm of a positive number to the base 2, e, or 10. A 7th-degree polynomial is utilized as the approximation method. `B3-11.0` uses an altered multi-argument LGP-21 function call sequence. The function argument must reside in the accumulator, but the two words immediately following the jump to the subroutine must contain the q of the argument and the desired base of the logarithm. The return value is in the accumulator on return.

STORAGE REQUIREMENTS

Program Space: 1 track, 58 words
Temporary Storage: None

INPUT

The required input consists of:

1. N: The positive number whose logarithm is to be calculated.
2. Q: The q factor of N.
3. K: The desired base of the logarithm.

N must be a positive number greater than zero ($N > 0$) and must reside in the accumulator with a positive q factor when branching to the subroutine.
The q factor must lie within the range $0 \leq q \leq 31$. In the calling sequence, Q is stored in the sector portion of a `Z` instruction that immediately follows the branch instruction.
K, the desired base of the logarithm, is also stored in the sector portion of a `Z` instruction.

- For $\log_2 N$, $K =$ `00`
- For $\log_e N$, $K =$ `01`
- For $\log_{10} N$, K can take any value other than `00` or `01`.

The `Z` instruction containing K is the final entry in the calling sequence.

CALLING SEQUENCE

```
α − 1   B   N       ; load N@1 into the accumulator
α       R   L+24    ; set return address
α + 1   U   L       ; call the log routine
                    ; The following parameter constants
                    ; are effectively scaled at @29
α + 2   Z   00Q     ; The q value of N
α + 3   Z   00K     ; Desired base K
```

As usual, any instruction sequence leaving N in the accumulator prior to the call is fine as long as it leaves N there with a positive q. The instruction at location α stores the address $(\alpha + 2)$ of the `Z` instruction containing Q into the subroutine's internal tracking mechanism, giving the subroutine the address of Q and K.[9].

Execution resumes following the second parameter constant upon return from the subroutine.

OUTPUT

Upon the return branch to the main program, the calculated logarithm to the desired base resides in the accumulator@6.

OPERATING DETAILS

1. **Accuracy:**
   Any potential error is less than 3×10^{-8}.

2. **Execution time:**
   Approximately 1350 ms.

3. **Program input:**
   The program tape contains the routine in two formats:

   - Relocatable hexadecimal, valid for J1-10.0 (section 8.1)
   - Relocatable decimal, in coding sheet format.

4. **Required input/output devices:**
   None.

5. **Overflow:**
   Overflow is neither checked upon entry to the subroutine nor reset upon exit.

[9]Translator's note: the subroutine figures out the return address by taking the address supplied by the `R` instruction and adding 2 to skip over the two parameter words. It then updates a `U` instruction in *itself* to jump back to the proper return address

6. **Error Stops:**

Memory Location	Meaning and Remedy
`main program+8`	Argument N is zero or negative. The accumulator contains the value $N-$(`1@30`). To continue, deposit a corrected value, and jump manually via the console to resume execution.

Translator's Notes

To a modern programmer accustomed to software exceptions and terminal logs, or one not quite as modern who's used to core dumps, the behavior of `B3-11.0` upon encountering an invalid argument ($N \leq 0$) might seem strange. The program halting at location `L+8` is clear enough; that leaves execution realtively close to where the error occurred. But why set the accumulator to $N - $(`1@30`)?

`1@30` denotes the integer value 1 scaled to a q factor of 30, which places a single `1` bit at Bit 30 (representing a quantitative value of 2^{-30}). This is the next-to-last usable bit in the word.

Let's say a programmer accidentally passed a negative fraction; for example, $N = -0.5$ at a q of 1. This value natively resides in the accumulator as:

`11000000000000000000000000000000`

When the error trap, occurs, the program subtracts 1@30 from the argument and leaves it in the accumulator. Subtracting a bit so low in the word forces a massive binary borrow chain to propagate all the way up to the first available `1` bit at the MSB side:

```
  11000000000000000000000000000000  (N = -0.5@1)
- 00000000000000000000000000000010  (1@30)
------------------------------------
  10111111111111111111111111111110  (Result remaining in Accumulator)
```

Remember that the LGP-21 doesn't have a big console with a lot of lights; it just has a small oscilloscope tube with a custom mask that displays the accumulator, the current address, and the instruction to be executed, painting a visual representation of the bits. A `0` bit is drawn at the baseline, while a `1` bit is drawn higher. The LGP-21 runs slowly enough that this does a pretty good job os showing the machine status continuously.

When this subroutine executes its error stop, the oscilloscope will be displaying a static image of the current contents of the accumulator. Reading from left to right (Bit 0 to Bit 31), the operator would see:

- A single **HIGH** dash at the far left edge (Bit 0, confirming a negative sign).
- A single **LOW** dash immediately following it (Bit 1, dropped to 0 by the borrow).

- An unbroken, massive **wall of 29 HIGH dashes** stretching across the screen (Bits 2 through 30, flipped to 1).
- A final **LOW** dash at the absolute right margin (Bit 31, remaining 0).

This striking visual pattern, a solid block of high dashes bracketed by a low one on the left, serves as an immediate, non-textual diagnostic flag. A seasoned operator could glance at the oscilloscope screen from across the room and instantly diagnose that the logarithm routine had been fed an illegal negative vector, making it unnecessary to print or punch memory contents to debug the issue.

4 Root Extraction

4.1 Square Root 1

B4-10.0

PURPOSE

The program `B4-10.0` calculates the square root of a positive number. The number must reside in the accumulator at an even q factor. The program utilizes Newton's method to solve the equation:

$$0 = x^2 - N \tag{4}$$

by means of successive approximations:

$$x_{i+1} = x_i + \frac{1}{2}\left(\frac{N}{x_i} - x_i\right) \tag{5}$$

The program operates as a subroutine and is invoked via a calling sequence in the main program. Upon branching to the subroutine, the argument must reside in the accumulator, and the return address must be stored within the subroutine. Upon return to the main program, the calculated square root is held in the accumulator.

STORAGE REQUIREMENTS

Program Space: 51 words
Temporary Storage: None

INPUT

The input variable for "Square Root 1" is N, the positive number whose square root is to be calculated.
N must reside in the accumulator with an even q factor when branching to the subroutine.

CALLING SEQUENCE

Location	Instruction	Address	Description
$\alpha - 1$	`B`	`L`	Bring N into the accumulator at an even q.
α	`R`	`L+50`	Store the return address in the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.

This function uses a standard LGP-21 function call. The only notable factor is that N must have an even q.[10]

[10] See for why.

OUTPUT

Upon the return branch to the main program, the square root of the argument N resides in the accumulator at $q_N/2$. For example, if N is `@24`, then $\sqrt{N}$ will be `@12`.

OPERATING DETAILS

1. **Accuracy:**
   No error is greater than 2^{-30}.

2. **Execution Time:**
   Approximately 775 ms.

3. **Program input:**
   The program tape contains the routine in two formats:

   - Relocatable hexadecimal, valid for J1-10.0 (section 8.1)
   - Relocatable decimal, in coding sheet format.

4. **Required Input/Output Units:**
   None.

5. **Overflow:**
   Overflow is ignored and is not reset upon return.

6. **Error Stops:**

Memory Location	Meaning and Remedy
`L+24`	N is negative. After pressing the START button, the accumulator is cleared, and control returns directly to the main program.

4.2 n-th Root

B4-11.0

PURPOSE

The program `B4-11.0` calculates the n-th root of any non-negative value. The exponent n must be an integer such that $2 \leq n \leq 20$. The results are calculated using an iterative method according to the formula:

$$y_{i+1} = y_i + \frac{1}{n}\left(\frac{A}{y_i^{n-1}} - y_i\right) \tag{6}$$

where y_i represents the i-th approximation of the root.
`B4-11.0` uses a calling sequence similar to **??**, except it has only one secondary parameter. The number A whose root is to be taken is loaded into the accumulator with an appropriate scale factor (see below), and the degree of the root n is stored in a `Z` instruction following the `R` that calls the subroutine. On return to the main program, the desired root resides in the accumulator@q/n.

STORAGE REQUIREMENTS

Program Space: 1 track (64 words)
Temporary Storage: Sectors 01, 02, 04, 05, 44, 53, 54, and 60 of track 63.

INPUT

The input variables for `B4-11.0` are A, the non-negative value whose root is to be calculated, and n, the integer specifying the degree of the root ($2 \leq n \leq 20$). Upon branching to the subroutine, A must reside in the accumulator scaled to a q factor that is exactly divisible by n.[11]

CALLING SEQUENCE

Location	Instruction	Address	Description
$\alpha - 1$	`B`	A	Bring A into accumulator at a q divisible by n.
α	`R`	`L+16`	Store the parameter address in the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.
$\alpha + 2$	`Z`	`08nn`	Parameter n encoded as a decimal sector address.

The instruction at location $\alpha - 1$ brings the value A into the accumulator at a q factor that is divisible by n. (Any instruction sequence that achieves this result is permissible.)

[11] This is similar to the requirement that a number whose square root is being taken is divisible by two; however, n only needs to divide A's q factor exactly to be acceptable. As an example, if we had A@12, then the square, cube, fourth, and sixth roots of A could all be taken, as all are even divisors of 12.

The instruction at location α stores the address (α+2) containing the parameter instruction for n within the subroutine, and execution resumes at $\alpha + 3$.[12]

OUTPUT

Upon the return branch to the main program, the n-th root of the argument resides in the accumulator scaled to a q factor of q_A/n. For example, if $n = 3$ and $q_A = 12$, the result $\sqrt[3]{A}$ will have a q of `@4`.

OPERATING DETAILS

1. **Accuracy:**
   The result is guaranteed accurate to eight decimal places.

2. **Execution time:**
   Between 2 and 60 seconds. Execution times are significantly longer when calculating high-degree roots of small values.

3. **Required input/output devices:**
   None.

4. **Overflow:**
   Overflow is ignored upon entry and is not reset upon exit.

5. **Error stops:**

Memory Location	Meaning and Remedy
`L+61`	The requested root is an imaginary value (i.e., an even-degree root of a negative number). Pressing the START button clears the accumulator and returns execution directly to the main program.

[12] The subroutine takes the parameter address supplied via the `R` instruction, recalculates the return address, and adjusts its own code to properly return.

5 Floating Point

5.1 Floating Point Interpreter 1

6 Floating-Point Interpretive Systems

6.1 Floating-Point Interpretive System 1

H1-11.0

PURPOSE

The Floating-Point Interpretive System 1 allows the programmer to prioritize floating-point calculations over standard fractional fixed-point operations. Data values are supplied to the system as 7-digit integers multiplied by a power of 10; the interpretive system automatically scales the data during calculation and delivers the final results as 7-digit integers multiplied by a power of 10.
Although this is fundamentally a single-word interpretive system, two memory words are required for each number during calculation and output operations. The mantissa and exponent are automatically separated and reassembled by the system. Consequently, a precise prior knowledge of the order of magnitude of a problem's variables is not required. The system features 33 distinct instructions, which significantly improves both programming ease and computational accuracy.
The interpretive system consists of various specialized subroutines—Data Input, Data Output, and the calculation of transcendental functions—which can be utilized either as a comprehensive program package or individually. These subroutines are invoked via specific interpretive instructions, rendering standard main-program calling sequences unnecessary. However, the interpretive system engine itself and the companion "Fixed-Point to Floating-Point Conversion (and vice versa)" routine are accessed via a traditional machine calling sequence.

DOCUMENTATION INDEX

This manual section contains comprehensive descriptions of the following subroutines and specialized features of the interpretive system:

Title	Program No.	Page
Internal Number Format	—	2
Internal Registers	—	2
Floating-Point Instructions	—	4
Data Input and Output 1	J9-10.0	8
Sine-Cosine 3	B1-10.2	10
Arctangent 2	B1-11.2	10
Logarithm 2	B3-11.1	10
Exponential Function 2	B3-10.1	10
Square Root 2	B4-10.1	10
Calling Sequence	—	10
Fixed-Point/Floating-Point Conversion	I3-12.0	11
Memory Allocation	—	12
Operating Instructions	—	13
Remarks	—	14
Comprehensive Example	—	15
Instruction Summary List	—	16

INTERNAL NUMBER FORMAT

Every floating-point number is held within memory inside a single cell as a split mantissa and exponent. Every mantissa is maintained in binary notation at a scaling factor of $q = 0$ and consists of an algebraic sign bit followed by 24 bits of precision. The most significant bit has a fractional value of 2^{-1}, meaning that all stored mantissas are strictly normalized.
The exponent is maintained at a scaling factor of $q = 30$ and consists of an algebraic sign bit and 5 bits of precision. The exponent represents the power-of-2 base by which the mantissa must be multiplied to obtain the actual represented numeric value. Expressed mathematically, the components represent the floating-point number as:

$$Z = M \cdot 2^E$$

Where:

— Z = The represented numerical value.

— M = The normalized binary fractional mantissa ($q = 0$).

— E = The binary integer exponent ($q = 30$).

For example, the value **3.75** would be represented in a computer memory word as:

`01111100000000000000000000000100`

Where:

— Mantissa ($q = 0$): $M = 0.9375$

— Exponent ($q = 30$): $E = 2$

— Verification: $Z = 0.9375 \cdot 2^2 = 3.7500$

The mantissa and exponent are treated by the system as two entirely distinct numbers. Therefore, a negative mantissa and a positive exponent can reside within the exact same memory word safely. The component parts of a number are complemented independently of one another.

For example, the negative value **-3.75** would be represented as:

`10001000000000000000000000000100`

Where:

— Mantissa ($q = 0$): $M = -0.9375$

— Exponent ($q = 30$): $E = 2$

— Verification: $Z = -0.9375 \cdot 2^2 = -3.7500$

As a further example, the fractional value **0.46875** in floating-point form would be represented as:

```
01111100000000000000000011111110
```

Where:

— Mantissa ($q = 0$): $M = 0.9375$

— Exponent ($q = 30$): $E = -1$

— Verification: $Z = 0.9375 \cdot 2^{-1} = 0.46875$

When values are loaded inside the simulated floating-point accumulator or the multiplication register, the numbers are structured so that they occupy **two discrete memory cells**, with the mantissa shifted one place to the right. Despite this physical expansion, the structural relationship between each mantissa and its respective exponent is rigorously maintained.

INTERNAL REGISTERS

Although every cell inside standard drum memory holds both a mantissa and its exponent packed together, two full words are required to hold a number during intermediate calculations and output processing: namely, one cell for the isolated mantissa and one cell for the exponent.

To facilitate data processing from memory, the interpretive system establishes a simulated floating-point accumulator, a multiplication register, an address register, and an instruction counter register. The accumulator and multiplication register each occupy two memory cells, while the address register and instruction counter occupy only one cell each. All data remains in floating-point format throughout processing. Operands and addresses can be modified dynamically without exiting the interpretive system environment.

The initial contents of all registers are zero when the program has just been freshly loaded and not yet executed. The contents of all registers (with the sole exception of the instruction counter) are preserved upon system switches and are modified exclusively by explicit interpretive instructions. Therefore, the user can exit and re-enter the interpretive system repeatedly without corrupting or altering the values held in the registers.

The instruction counter is set upon every entry into the interpretive system and is incremented during the execution of every instruction. These specialized registers are detailed below:

1. The Floating-Point Accumulator The floating-point accumulator holds intermediate calculation results. Although the mantissa is stored in a normalized state in standard memory, it is shifted right by one binary place when held inside the floating-point accumulator. This means the mantissa sits at $q = 0$ with its most significant bit shifted one position to the right; the exponent sits at $q = 30$. Both components retain their independent algebraic sign bits. The

contents of the accumulator are altered exclusively by arithmetic operations, function evaluations, or via the instructions: “Clear”, “Swap”, “Set Sign”, or during a function calculation.

2. The Multiplication Register The multiplication register (M-register) holds the multiplicand during the execution of the “Multiply” and “Cumulative Multiplication” instructions. The mantissa sits at $q = 0$ with its most significant bit shifted one position to the right; the exponent sits at $q = 30$. Both parts are provided with their respective algebraic signs. The contents of this register are modified exclusively by the “Set” and “Swap” instructions.

3. The Address Register The address register consists of a single cell holding a track/sector value scaled at $q = 29$. This register is utilized to dynamically modify the operand address portion of standard instruction words in memory via the “Store Address” command. Additionally, this register is used during “Test for Zero” branch operations. The contents of this register can be modified exclusively by the “Set Address” or “Increment Address” instructions.

4. The Instruction Counter Register The instruction counter register tracks the memory address of the floating-point instruction currently being executed and flags the address of the next instruction to be processed. When executing a “Return” instruction, the operand address portion of the target cell is overwritten with the current contents of the instruction counter plus 2. The register is initially seeded by the main program’s calling sequence and increments by 1 with each instruction executed. This sequential operation is broken only by explicit jump instructions, which overwrite the register with a new target address.

FLOATING-POINT INSTRUCTIONS

This interpretive system operates using 33 pseudo-instructions. Each instruction consists of an operation code field and an operand address field. Certain operand addresses carry a divergent structural meaning compared to native fixed-point machine commands (e.g., null/zero addresses).
The floating-point instructions are detailed below. The placeholder notation `ttss` denotes an operand address written in decimal track and sector format. All referenced registers represent the simulated virtual registers of the interpretive engine. All instructions execute to completion without exiting the interpretive system.

1. Arithmetic Instructions

   `B ttss` **(Bring):** Copies the contents of cell `ttss` into the floating-point accumulator. The source memory remains unchanged.

`A ttss` **(Add):** Adds the contents of cell `ttss` to the current value of the accumulator. The sum replaces the contents of the accumulator. Memory remains unchanged.

`S ttss` **(Subtract):** Subtracts the contents of cell `ttss` from the current value of the accumulator. The difference replaces the contents of the accumulator. Memory remains unchanged.

`D ttss` **(Divide):** Divides the current value of the accumulator by the contents of cell `ttss`. The resulting quotient replaces the contents of the accumulator. Memory remains unchanged.

`P ttss` **(Set M-Register):** Copies the contents of cell `ttss` into the multiplication (M) register. Memory remains unchanged.

`M ttss` **(Multiply):** Multiplies the contents of the M-register by the contents of cell `ttss`. The product replaces the contents of the accumulator. The source memory and the M-register remain unchanged.

`N ttss` **(Cumulative Multiplication):** Multiplies the contents of the M-register by the contents of cell `ttss`, and automatically adds the product directly to the current accumulator value. The final sum resides in the accumulator. Memory and the M-register remain unchanged.

`D 000y` **(Shift Right):** Divides the current contents of the accumulator by 2^y. The operand address must be formatted as `000y`, where $0 \leq y \leq 9$. The accumulator contents remain strictly in floating-point format.

`M 000y` **(Shift Left):** Multiplies the current contents of the accumulator by 2^y. The operand address must be formatted as `000y`, where $0 \leq y \leq 9$. The accumulator contents remain strictly in floating-point format.

`H ttss` **(Hold):** Copies the current contents of the accumulator into memory cell `ttss`. The accumulator contents remain unchanged.

`C ttss` **(Clear):** Copies the current contents of the accumulator into memory cell `ttss`, and subsequently flushes the accumulator register entirely with zeros.

2. Jump / Branch Instructions

`U ttss` **(Unconditional Jump):** Overwrites the instruction counter register with the address `ttss`. Execution loops to fetch the next instruction from cell `ttss`. This command cannot be used to exit the interpretive system.

`T ttss` **(Test for Minus):** Overwrites the instruction counter register with address `ttss` if the current accumulator value is negative; execution then branches to `ttss`. If the accumulator is positive or zero, the branch is ignored and the next sequential instruction is executed. This command cannot be used to exit the interpretive system.

`800T ttss` **(Conditional Jump):** Overwrites the instruction counter register with address `ttss` if the accumulator value is negative OR if Program Selector Switch 7 (`PST`) is turned ON. Otherwise, execution continues sequentially. This command cannot be used to exit the interpretive system.

3. Address Modification Instructions

   `E ttss` **(Set Address):** Extracts the operand address portion of the instruction word stored at cell `ttss` and copies it into the simulated address register. Memory remains unchanged.

   `I ttss` **(Increment Address):** Increments the current numeric value of the simulated address register by the value `ttss`. This instruction can be used to decrement the register by supplying the complement of the value `ttss`.

   `Y ttss` **(Store Address):** Copies the tracking address currently held in the simulated address register into the operand address field of the instruction word at cell `ttss`. The remaining fields of cell `ttss` and the address register itself are unaltered.

   `Z ttss` **(Test for Zero):** Subtracts the literal value `ttss` from the current contents of the simulated address register. If the result is non-zero, the immediately following sequential instruction is executed. If the result is exactly zero, the next instruction is skipped and execution jumps directly to the second following word (the instruction after next).

   `R ttss` **(Return):** Computes the current value of the instruction counter register plus 2, and writes that calculated address into the operand address field of the instruction word at cell `ttss`. The instruction counter itself and the remaining fields of cell `ttss` are untouched.

4. Auxiliary Instructions The operand address field for all auxiliary commands must be exactly zero (`0000`).

   `U 0000` **(Swap):** Swaps the contents of the multiplication (M) register and the floating-point accumulator.

   `B 0000` **(Set Plus Sign):** Forces the accumulator value to be positive if it is currently negative. If the accumulator is already positive or zero, the command is skipped as a no-operation.

   `T 0000` **(Set Minus Sign):** Forces the accumulator value to be negative if it is currently positive. If the accumulator is already negative, the command is skipped as a no-operation.

   `Y 0000` **(Invert Sign):** Replaces the accumulator contents with its arithmetic complement (negating the current sign).

   `Z 0000` **(Stop):** Halts all computation loops immediately, provided that physical Break Point Switch 16 (`PS-16`) is turned OFF. Pressing the

physical "Start" button on the console resumes execution at the next sequential memory cell. If `PS-16` is turned ON, this stop command is bypassed entirely.

`E 0000` **(Exit):** Terminates execution within the interpretive engine. The instruction immediately following this command word is decoded and executed as a standard native machine instruction.

5. Input / Output Instructions The operand address field for all I/O commands must be exactly zero (`0000`).

   `I 0000` **(Input):** Reads a block of data from tape, automatically parses it into floating-point format, and stores it in sequential memory addresses for main-program access. The target destination base address is read directly from the data tape leader header.

   `P 0000` **(Output):** Prints the current contents of the floating-point accumulator to the output unit in standard floating-point notation. The accumulator state is completely preserved during the print operation.

6. Transcendental Function Evaluation Instructions The operand address field for all function evaluations must be exactly zero (`0000`).

   `R 0000` **(Square Root):** Computes the square root ($\sqrt{x}$) of the current accumulator contents. The result replaces the value in the accumulator.

   `S 0000` **(Sine):** Calculates the sine ($\sin x$) of the current accumulator contents. The result replaces the value in the accumulator. The input argument x must be supplied strictly in radians.

   `C 0000` **(Cosine):** Calculates the cosine ($\cos x$) of the current accumulator contents. The result replaces the value in the accumulator. The input argument x must be supplied strictly in radians.

   `A 0000` **(Arctangent):** Calculates the principal value of the arctangent ($\arctan x$) of the current accumulator contents. The resulting angle is returned strictly in radians.

   `N 0000` **(Natural Logarithm):** Calculates the natural logarithm ($\ln x$, base x relative to e) of the current accumulator contents.

   `H 0000` **(Exponential Function):** Computes the natural exponential value (e^x), where x represents the initial numeric value held in the accumulator.

Data Input and Output 1

Data Input

The "Data Input and Output 1" subroutine introduces data into the computer for utilization by a main program. Decimal numbers are read from tape, au-

tomatically converted into binary floating-point format, and stored into the specified memory cells in the required configuration.

This subroutine is invoked directly via an `I 0000` pseudo-instruction processed within the interpretive loop of the main program. Control automatically returns to the memory cell immediately following this input command once all data elements have been successfully processed and stored.

The input data tape must contain the following expressions in the exact order and format described below:

a) Input Keyword (Control Word) The first expression in each distinct group of data elements must be a control word that specifies both the number of decimal places P for every word within that group, and the base starting address `ttss` of the destination memory block.

   The parameter P must be supplied as a two-digit decimal integer preceded by an algebraic sign, constrained strictly within the range:

   $$-03 \leq P \leq +16$$

   The starting destination address must be written as an absolute address in decimal track and sector notation (`ttss`). A positive P value indicates that the decimal point is shifted to the left; a negative P value indicates that the decimal point is shifted to the right.

b) Numeric Data Elements The individual data entries are represented on tape as signed decimal integers containing up to 7 digits of precision. For positive values, both the explicit plus sign and any leading zeros may be omitted entirely. For negative values, however, all leading zeros between the minus sign and the first significant digit must be explicitly punched on the tape. A value of zero requires only a single stop code (`'`) to be written.

   Example: Given an input control keyword punched as `+032400'`, the fractional decimal value **16.192** would be encoded and supplied on tape simply as `16192'`.[13]

c) End-of-Block Keyword and Minus-Zero Termination The end of a specific data group is flagged on tape by an explicit minus-zero control word, formatted as `-0000000'`. This tracking word functions strictly as a structural boundary indicator and is not saved in memory by the subroutine.

   The definitive end of the entire reading operation is signaled by appending a second consecutive stop code immediately following this minus-zero control word (i.e., formatted as `-0000000''`).

   When the interpreter reads the singular minus-zero word (`-0000000'`), the program automatically transitions to read the next sequential input

[13] **Translator's Note:** Because $P = +03$, the interpreter shifts the implied decimal point 3 places to the left upon ingestion, reconstructing $16192 \times 10^{-3} = 16.192$.

control keyword on the tape, subsequently converting all following data values according to the newly defined P scaling factor and storing them starting at the new destination base address.

When the double-stop code for the absolute end of the print/read sequence (`-0000000''`) is encountered instead, the subroutine executes a carriage return and returns execution control back to the main program.

The following example illustrates the physical layout of a typical structured data block:

Input control word			Sto	Decimal numbers with sign		Sto	
±	P	Cell	p	±		p	CR
+	07	2004	’		1090000	’	☐
					120000	’	☒
					340000	’	☐
					560000	’	☒
					780000	’	☐
					900000	’	☒
+			’	–	0000000	’	☐
+	03	2010	’	–	4320001	’	☒
				–	0000021	’	☐
				–	3470	’	☒
				–		’	☐
					10	’	☒
				–	0000000	’	☐
						’	☒

Table 1: LGP-21 Floating Point Tape Input Example

The first six decimal numbers are read in and stored as follows:

Memory Cell	**Contents**
2004	109
2005	0.012
2006	0.034
2007	0.056
2008	0.078
2009	0.09

The second record group consisting of five data words would be stored as follows:

Memory Cell	Contents
2010	−4320.001
2011	−0.021
2012	3.47
2013	0
2014	0.01

The minus-zero control word `-0000000'` terminates each individual group and triggers the system to read the next record block. All mantissas are maintained at a binary scaling factor of $q = 0$ with a precision length of 24 bits. All exponents are stored within the exact same memory word as their respective mantissa, scaled at $q = 30$. The additional tracking stop code appended behind the second minus-zero control word completely terminates the data input sequence.

2. Data Output

The "Data Input and Output 1" subroutine delivers the contents of the floating-point accumulator as a signed, seven-digit decimal mantissa and a signed, two-digit decimal exponent. Following the output execution, a tabulator character (tab space) is issued, and control branches back to the memory cell immediately following the `P 0000` instruction that originally invoked the subroutine.

Output Format The printed output expression uses the following structural form:

`.XXXXXXX`± `EE`±

Where:

— `.XXXXXXX`± represents the decimal mantissa with its trailing sign.

— `EE`± represents the power-of-10 base exponent with its trailing sign.

Output Examples The following table illustrates the raw machine printout format paired with its conventional decimal representation:

Machine Printout	Conventional Representation
`.5060000- 04`	−5060.000
`.0937500- 02-`	−0.000937500
`.1000000- 06-`	−0.0000001000000
`.1000000- 10`	-0.1×10^{-10}

SINE-COSINE 3, Program B1-10.2

"Sine-Cosine 3" calculates the sine or cosine of an argument provided in floating-point radians. Upon returning to the main program, the result is stored in the floating-point accumulator. The return is executed to the cell immediately following the `S0000` or `C0000` instruction, depending on which subroutine was invoked.

ARCTANGENT 2, Program B1-11.2

"Arctangent 2" calculates the arctangent of an argument located in the floating-point accumulator. The result is placed in the floating-point accumulator in radians when control returns to the cell of the main program following the `A0000` instruction.

LOGARITHM 2, Program B3-11.1

"Logarithm 2" calculates the natural logarithm (base e) of a number located in the floating-point accumulator. The result remains in the floating-point accumulator upon return to the cell of the main program following the `N0000` instruction.

EXPONENTIAL FUNCTION 2, Program B3-10.1

"Exponential Function 2" calculates the value e^x, where x is the contents of the floating-point accumulator. The result is stored in the floating-point accumulator upon return to the cell of the main program following the `H0000` instruction.

SQUARE ROOT 2, Program B4-10.1

"Square Root 2" calculates the square root of a positive number located in the floating-point accumulator. The result is placed in the floating-point accumulator upon return to the cell of the main program following the `R0000` instruction.

CALLING SEQUENCE

The "Floating-Point Interpreter System 1" is invoked via a calling sequence from the main program. Prior to branching into the system, the starting address of the floating-point instructions to be interpreted must be supplied to the system. The floating-point instructions must immediately follow the branch instruction. An example is provided below:

Cells α and $\alpha+1$ contain instructions `R` and `U`, which store the target address (α+2) into the interpreter system and execute a branch to L, the starting address of the interpreter system. Cells $\alpha+2$ through $\alpha+n$ contain the actual floating-point instructions. An explicit exit from the interpreter system is required when, as shown here at $\alpha+n+1$, fixed-point machine instructions follow.

Memory Cell	Instruction	Address	Description
α	R	L_0	Store starting address of floating-point instructions.
$\alpha + 1$	U	L_0	Entry into the interpreter system.
$\alpha + 2$	...		Floating-point instructions.
$\alpha + n$	...		
$\alpha + n + 1$	XE	0000	Exit from the interpreter system. Return to fixed-point instructions.

FIXED-POINT TO FLOATING-POINT CONVERSION (AND VICE VERSA), L3-12.0

This program serves two functions. It converts a fixed-point number into the floating-point format defined for "Floating-Point Interpreter System 1", or it converts a floating-point number back into a fixed-point number. This program is not an integral part of the interpreter system core; rather, it is a fixed-point subroutine designed to convert data intended for, or generated by, the interpreter system.

Before executing the calling sequence for this subroutine, the interpreter system must be exited. The number to be converted must reside in the hardware accumulator, and its scale factor q must be supplied by the calling sequence.

When converting back, a floating-point number must not be too large to be held at the specified q. When converting forward, a fixed-point number must not require an exponent greater than 31.

Upon return to the main program, the converted number resides in the machine accumulator. The value will either be in fixed-point form at the q provided by the calling sequence (if converted back), or in floating-point form corresponding to the specified q (if converted forward).

The respective calling sequences are explained below. The first is for converting a number from fixed-point to floating-point, and the second is for converting a number from floating-point back to fixed-point. Only one operation per number can be executed during a single call.

Conversion to floating-point

Memory Location	Instruction	Address	Description
$\alpha - 1$	B	[ ]	Load fixed-point number into accumulator.
α	R	L+25	Store address of q.
$\alpha + 1$	U	L	Entry into the subroutine.
$\alpha + 2$	Z	$089q$	Specify q of the number.
$\alpha + 3$	etc.		Return to main program.

The instruction at $\alpha - 1$ loads the fixed-point number into the accumulator (any instruction achieving this may be used). The instruction at α stores the address $(\alpha + 2)$ of the value q into the subroutine. The instruction at $\alpha + 1$ causes a branch to L, the starting address of this subroutine. The instruction at $\alpha + 2$ contains the q of the fixed-point number formatted as a decimal sector address. The return from the subroutine occurs at $\alpha + 3$.

Conversion to fixed-point

Memory Location	Instruction	Address	Description
$\alpha - 1$	B	[ ]	Load floating-point number into accumulator.
α	R	L+148	Store address of q.
$\alpha + 1$	U	L+122	Entry into the subroutine.
$\alpha + 2$	Z	$08qq$	Specify target q for the number.
$\alpha + 3$	etc.		Return to main program.

The instruction at $\alpha - 1$ loads the floating-point number into the primary hardware accumulator (any instruction achieving this is permissible). The instruction at α stores the address $(\alpha + 2)$ of the value "q" into the subroutine. The instruction at $\alpha + 1$ causes a branch to the appropriate entry cell, $L + 122$, of this subroutine. The instruction at $\alpha + 2$ contains the "q" that the converted fixed-point number should assume, formatted as a decimal sector address. The return from the subroutine occurs at $\alpha + 3$.

PROGRAM STOPS

There are two dedicated program stops within this subroutine. They are listed and explained below. L represents the starting address of this subroutine.

Memory Cell	Meaning and Remedy
$L + 152$	The number is too large for the specified "q" during backward conversion. Pressing "Start" clears the accumulator and returns control to the main program.
$L + 262$	The number requires an exponent greater than 31 during forward conversion. Pressing "Start" clears the accumulator and returns control to the main program.

Note that this subroutine is utilized independently. An input or output unit is not required. Storage requirements are detailed below.

STORAGE REQUIREMENTS

Only "Square Root 2" is natively bundled inside the actual interpreter system core. The other subroutines are provided individually. They can be loaded partially or entirely depending on application needs. The subroutines are listed below alongside their relative starting addresses and required memory footprints (where L represents the starting address of the interpreter system):

Subroutine	**Initial Filler Keyword**	**Modifier Keyword**	**Memory Required**
Interpreter System	L	L	10 Tracks
Data Input & Output 1	L+1000	L+1000	7 Tracks
Sine-Cosine 3	L+1700	L+1700	2 Tracks, 32 Sectors
Arctangent 2	L+1932	L+1932	1 Track, 32 Sectors
Logarithm 2	L+2100	L+2100	1 Track
Exponential Function 2	L+2200	L+2200	2 Tracks
Fixed/Floating Conversion & Vice Versa 1	*Anywhere*	*Anywhere*	3 Tracks

Track 63 is utilized by various parts of the system as an intermediate scratchpad memory, with the sole exception of "Fixed-Point to Floating-Point Conversion and Vice Versa 1", which requires no scratchpad storage. Therefore, no part of the system should ever be stored in Track 63.

OPERATION

1. Program Input

The "Floating-Point Interpreter System 1" is provided on two separate paper tapes. The first tape contains the program in an arbitrarily relocatable hexadecimal format, compatible with Program Input 1 (`J1-10.0`). The second tape contains the program in an arbitrarily relocatable decimal format structured in coding sheet layout. Each tape contains the interpreter system core followed by all subroutines in the exact order specified under "Storage Requirements". It is assumed that Program Input 1 starts at address `0000`.

If program selector switch PS-32 is turned ON, the relocatable hexadecimal program tape can be read continuously without manual intervention until the "Fixed-Point to Floating-Point Conversion and Vice Versa" routine is reached. If PS-32 is turned OFF, the computer will halt after each successfully read subroutine. (No stop is hardcoded between the Interpreter System core and "Data Input and Output 1".)

In either case, entering a new modifier keyword and a select keyword for the input device is strictly required before loading "Fixed-Point to Floating-Point Conversion and Vice Versa". This specific program must *never* be loaded using the same modifier as any part of the interpreter system core.

The relocatable decimal tape has a hardcoded stop programmed after the interpreter system core and after every individual subroutine. In all cases when using this tape, a new modifier and a new device selection keyword must be entered manually for each subroutine to be read.

For both tape types, whenever a stop occurs due to PS-32 OFF, the operator must press the "Start" button to return execution to Program Input 1 for subsequent data entry. If a stop occurs while PS-32 is ON, the computer will halt directly on an input command within Program Input 1.

2. Required Input/Output Units

The Flexowriter must be selected for both input and output operations.

3. Program Selector Switches

Switch	Setting	Function
PS-16	ON	Suppresses the programmatic halt at a `Z 0000` instruction.
	OFF	Permits a programmatic halt at a `Z 0000` instruction.
PS-32	ON	Forces an unconditional branch upon encountering an `-T` instruction[14]
	OFF	Forces a conditional branch upon encountering an `-T` instruction.

4. Program Stops

L represents the absolute starting address of the interpreter system.

REMARKS

The initial location of the interpreter system should ideally align with Sector `00` of a track so that the reference addresses targeting Track 63 are optimally mapped.

Remember that when the interpreter is running all instructions are pseudo-instructions! The interpreter is substituting itself for the standard fetch-execute cycle that the CPU would do. Another way of saying it is that the interpreter is an emulator for a CPU that has a floating-point instruction set, and we have an agreed-upon protocol to enter and exit emulation.

Location	Meaning	Remedy
$L + 557$	The exponent is too large for an `H` or `C` instruction, or the argument of a square root is negative. The address of the offending instruction resides in the machine accumulator.	Press "Start" to bypass. The `H` instruction, `C` instruction, or square root calculation will not be executed.
$L + 759$	Division by an illegal value (Division by Zero).	Do not proceed. The data value must be corrected.
$L + 1152$	The input value possesses an exponent that is too large.	Press "Start" to advance to the next pseudo-instruction. A value of zero is stored in place of the rejected number.
$L + 2005$	The argument for a natural logarithm is ≤ 0.	Press "Start" to proceed with a computed result of zero.
$L + 2056$	The exponent is too large for a natural logarithm operation.	Do not proceed. The data value must be corrected.

Pseudo-instructions utilizing addresses from `0000` to `0009` have special meanings. When pseudo-instructions use these addresses, they do not refer to the actual storage at these addresses, but instead invoke functions in the interpreter. The interpreter utilizes 16 such special instructions, including `D 000y` and `M 000y`. Although instructions `D 000y` and `M 000y` may carry a literal value of y between 1 and 9, standard instructions like `D ttss` and `M ttss` (while in interpreter mode) cannot reference any cell between `0000` and `0009`[15].

EXAMPLE

The following sample program demonstrates how floating-point instructions are structured for the interpreter system, and how input data must be formatted to achieve the desired output.

Given a fourth-degree polynomial of the form:

$$Ax^4 + Bx^3 + Cx^2 + Dx + E = y$$

The value of y is to be calculated and printed.

The Interpreter System is assumed to be stored starting at address `0300`. The main program is stored starting at address `4200`. The data values are stored sequentially starting at cell `4306`.

[15]The addresses from `0000` to `0009`, inclusive, trigger transcendental functions, etc. *when the floating-point interpreter is running.* In normal execution mode, they're used just the same as any other location in the machine.

PROGRAMMING EXAMPLE

This example demonstrates how to evaluate a fourth-degree polynomial using the Floating-Point Interpreter System's extended instruction set. The polynomial is structured as:

$$y = Ax^4 + Bx^3 + Cx^2 + Dx + E$$

By applying Horner's Method, the equation is factored to minimize arithmetic operations, reducing it to a sequence of successive multiplications and additions:

$$y = (((Ax + B)x + C)x + D)x + E$$

The Interpreter System core is loaded at address `0300`. The main program begins at cell `4200`. The five floating-point coefficients (A, B, C, D, E) and the variable x are stored as 3-word blocks starting at cell `4306`.

Loc	Op	Addr	Remarks / Description
4200	R	0300	; Set up return link pointer for interpreter loop
4201	U	0300	; Enter Interpreter Mode (Jumps to system core)
4202	B	4306	; Load coefficient A into pseudo-accumulator
4203	M	4324	; Multiply by x [Ax]
4204	A	4309	; Add coefficient B [Ax + B]
4205	M	4324	; Multiply by x [(Ax + B)x]
4206	A	4312	; Add coefficient C [(Ax + B)x + C]
4207	M	4324	; Multiply by x
4208	A	4315	; Add coefficient D
4209	M	4324	; Multiply by x
4210	A	4318	; Add coefficient E [Final polynomial value y]
4211	H	4321	; Store calculated result y into memory block
4212	XE	0000	; Exit Interpreter Mode (Return to native hardware)
4213	H	0000	; Native machine execution halt

2. Data Memory Allocation Map

Because the pseudo-accumulator manages multi-word operations, each floating-point literal spans 3 consecutive words on the magnetic drum.

Start Address	Symbol	Allocation Footprint
4306	A	Occupies cells 4306, 4307, 4308
4309	B	Occupies cells 4309, 4310, 4311
4312	C	Occupies cells 4312, 4313, 4314
4315	D	Occupies cells 4315, 4316, 4317
4318	E	Occupies cells 4318, 4319, 4320
4321	Y	Result storage (cells 4321, 4322, 4323)
4324	X	Input variable x (cells 4324, 4325, 4326)

QUICK REFERENCE INSTRUCTION SUMMARY

Operational Notation Definitions

( )	Contents of memory cell or register
→	Displaces / moves to / becomes
Acc	Floating-Point Software Accumulator
A Reg.	Address Register
M Reg.	Multiplier Register
C Reg.	Counter Register

Extended Interpreter Instruction Set

Op	Operand (ttss)	Result / Action
B	ttss	(ttss) → Acc
A	ttss	(Acc) + (ttss) → Acc
S	ttss	(Acc) - (ttss) → Acc
D	ttss	(Acc) / (ttss) → Acc
P	ttss	(ttss) → M Reg.
M	ttss	(M Reg.) × (ttss) → Acc
N	ttss	(M Reg.) × (ttss) + (Acc) → Acc
H	ttss	(Acc) → ttss
C	ttss	(Acc) → ttss; then 0 → Acc
U	ttss	Unconditional Jump: Next instruction is at ttss
T	ttss	Conditional Jump: Jump to ttss if (Acc) < 0
-T	ttss	Conditional Jump: Jump to ttss if (Acc) < 0 OR if PST is ON
E	--	Exit Interpreter Mode
I	ttss	ttss + (A Reg.) → A Reg.
Y	ttss	(Address Part of ttss) → A Reg.
Z	ttss	(A Reg.) → Address Part of ttss
R	ttss	Compare: If (A Reg.) - ttss = 0, skip next instruction; else loop

Special Behavior for Null Addresses

When the address field `ttss` of an interpreter command equals `0000` (or a designated function selector ID within Track 00), the default memory interaction is overridden to perform specialized administrative or algorithmic operations rather than referencing physical memory cell zero.

Interpreter Behavior for Null Addresses (Address = 0000)[16]

[16]Excluding extended subroutine selection commands under the format `D000y` or `M000y`. See Section 0479.

Op	Result / Action Executed (When Address Field is 0000)	
D	$(\text{Acc})^2 \rightarrow \text{Acc}$	[Square the Accumulator]
M	$(\text{Acc}) \times 2 \rightarrow \text{Acc}$	[Double the Accumulator]
U	Swap values: (Acc) $\longleftrightarrow$ (M Reg.)	
B	Make (Acc) positive	[Absolute Value: $\lvert Acc \rvert$]
T	Make (Acc) negative	[Force Negative Sign]
Y	Invert Sign: Change sign bit of (Acc)	
Z	Conditional Program Stop (Halts execution if console switch PS-16 is OFF)	
E	Exit Interpreter Mode (Return control to native machine hardware)	
I	Read floating-point numbers from paper tape input	
P	Print/Punch floating-point numbers to output device	
R	$\sqrt{(\text{Acc})} \rightarrow \text{Acc}$	[Square Root]
S	$\text{Sine}(\text{Acc}) \rightarrow \text{Acc}$	
C	$\text{Cosine}(\text{Acc}) \rightarrow \text{Acc}$	
A	$\text{Arctangent}(\text{Acc}) \rightarrow \text{Acc}$	
N	$\ln(\text{Acc}) \rightarrow \text{Acc}$	[Natural Logarithm]
H	$e^{(\text{Acc})} \rightarrow \text{Acc}$	[Exponential Function]

6.2 Floating Point Interpreter 2

H1-11.1

1. INTRODUCTION

The "Floating-Point Interpreter System 2" permits the programming of LGP-21 problems using floating-point arithmetic. The system accepts 9-digit whole decimal numbers as input data, scaled by a two-digit decimal exponent. It automatically handles all required scaling modifications during calculation and outputs results as a 7-digit signed decimal mantissa, followed by a 2-digit decimal exponent.

Because the system autonomously processes values across an exceptionally wide dynamic range, the programmer does not require precise prior knowledge of a problem's numerical boundaries. This structural capacity guarantees a high degree of calculation and output accuracy. Furthermore, this program expands the standard 16-instruction hardware set of the computer to **33 instructions**, making operations as simple and user-friendly as possible.

The "Floating-Point Interpreter System 2" is available as a single paper tape in an arbitrarily relocatable hexadecimal format, compatible with Program Input 1 (`J1-10.0`). It bundles various subroutines—such as Data Input, Data Output, and a suite of transcendental math functions—which are detailed later in this manual. Only the specific subroutines required for a given problem need to be read into memory alongside the main "Floating-Point Interpreter System 2" core; however, the memory cross-references for the entire system must be strictly preserved during loading.

The programmer will quickly find that utilizing this system yields a massive savings of time and effort when dealing with problems where data values

fluctuate wildly in magnitude—scenarios that would otherwise demand an immense, complex programming overhead under traditional fixed-point arithmetic constraints.

Principles

The "Floating-Point Interpreter System 2" intercepts and reinterprets the elementary hardware instruction structure of the LGP-21, transforming it so that it can be programmed exactly like a natively "hardwired" floating-point computer. This illusion is achieved through:

1. **Virtual Registers:** The provision of an internal, software-driven Multiplier Register, Address Register, and a dedicated Floating-Point Accumulator.
2. **Advanced Operations:** Integrated support for cumulative multiplication (multiply-accumulate), sign inversion, and direct function-generating commands.
3. **Massive Numerical Range:** Input data consists of signed 9-digit decimal numbers followed by a base-10 exponent between +99 and −99. Output data delivers signed 7-digit decimal numbers followed by a base-10 exponent between +99 and −99.
4. **Expanded I/O framework:** A vastly modernized structure for data entry and terminal printing commands.

The entire system footprint consumes **28 Tracks** of memory. This is divided into the core interpreter loop, Data Input (7 tracks), Data Output (4 tracks), and Floating-Point Functions (6 tracks). This leaves **36 Tracks (2,304 words)** completely free on the magnetic drum for the programmer's main execution routine and data storage arrays.

2. THE REGISTERS

The floating-point format requires two consecutive memory cells to represent a single data word. Consequently, the interpreter system features two specialized registers designed for data transport, processing, and output while strictly preserving the floating-point structure. These registers—the Floating-Point Accumulator and the Multiplier Register—each occupy two memory cells.

Address modification can be executed directly within the interpreter system by utilizing a simulated, single-cell Address Register. Additionally, a single-cell Counter Register governs the operational execution flow of the interpreter system loop.

Upon initial system loading, before any operations have commenced, the contents of all virtual registers are initialized to zero. A detailed breakdown of these specialized registers follows below. (The individual instructions referenced here are explained comprehensively in Section 4, "Pseudo-Instructions").

1. The Floating-Point Accumulator

The Floating-Point Accumulator holds intermediate mathematical results and output values. Because output data must ultimately be rendered as a signed 7-digit decimal mantissa with a 2-digit decimal exponent, the hardware treats them internally as follows:

- **The Mantissa:** Maintained within the Floating-Point Accumulator with a precision of 30 bits at binary scaling factor $q = 0$. The most significant bit possesses a weight of 2^{-1}, meaning the mantissa is *always* strictly normalized.
- **The Exponent:** Positioned at binary scaling factor $q = 29$.

Both the mantissa and the exponent maintain their own independent, algebraic sign bits. The contents of the Accumulator are altered exclusively by arithmetic operations, transcendental function calls, or explicit register management commands (Clear, Swap, or Set Sign).

2. The Multiplier Register

The Multiplier Register stores the multiplicand during standard multiplication and cumulative multiplication (multiply-accumulate) routines. * The internal mantissa is mapped at $q = 0$. * The internal exponent is mapped at $q = 29$.

Like the accumulator, both components carry their respective algebraic sign bits. The contents of this register can only be modified by executing explicit "Set" or "Swap" commands.

3. The Address Register

The Address Register holds a single virtual memory address formatted in Track/Sector notation at binary scaling factor $q = 29$. This register is specifically utilized to modify the operand address field of memory words via the "Store Address" command, and to govern main program execution loops during zero-testing routines. The contents of this register are altered exclusively by an explicit "Set Address" or "Increment Address" instruction.

4. The Counter Register

The Counter Register functions as the virtual Program Counter (PC). It tracks the absolute address of the currently executing pseudo-instruction and points ahead to the address of the subsequent instruction.

When a subroutine return sequence is initiated, the current contents of this register plus 2 (PC+2) are injected into the operand address field of the targeted return command. The register is initially primed by the main program's calling sequence and automatically increments by 1 following the execution of each pseudo-instruction. This linear, step-by-step increment can only be overridden by a programmatic branch or jump instruction.

INTERNAL NUMBER FORMAT

Every floating-point number requires two consecutive memory cells for its complete representation.

- **The first cell** holds the mantissa of the number in binary representation scaled at $q = 0$. Each mantissa consists of an algebraic sign bit and 30 significant binary digits; the most significant bit (MSB) carries a value of 2^{-1}. The mantissa is therefore always maintained in a strictly normalized state.
- **The second cell** holds the base-2 exponent, which must be multiplied by the mantissa to obtain the actual value of the represented number. This exponent, maintained at $q = 29$, is a whole binary integer carrying its own algebraic sign bit.

Taken together, these two components represent a floating-point number via the following mathematical relationship:

$$Z = M \cdot 2^E$$

Where Z = The Number, M = The Mantissa, and E = The Exponent.

Example 1: Representation of $+3.75$

In floating-point memory, the decimal value 3.75 is internally normalized as $0.9375 \cdot 2^2$. Its exact binary equivalent on the magnetic drum is structured as follows:

First Cell (Mantissa at $q = 0$):

Bit 0	Bits 1--30	Bit 31
0	11110000000000000000000000000000	0
↑	↑	↑
Sign (+)	Normalized Mantissa (0.9375)	Unused

Second Cell (Exponent at $q = 29$):

Bit 0	Bits 1--29	Bits 30--31
0	00000000000000000000000000000	10
↑	↑	↑
Sign (+)	Padding Zone	Binary Exponent (2)

Example 2: Representation of a Negative Number (-3.75)

A negative number is represented by inverting the leading sign bit of the mantissa cell:

First Cell (Mantissa at $q = 0$):

Bit 0	Bits 1--30	Bit 31
1	00010000000000000000000000000000	0
↑	↑	↑
Sign (−)	Two's Complement Mantissa	Unused

Sequential Allocation in Drum Memory

The continuous storage of alternating mantissas and exponents across sequential drum tracks steps through memory in fixed increments of two:

Memory Location	Cell Contents
4304	Mantissa of Variable 1
4305	Binary Exponent of Variable 1
4306	Mantissa of Variable 2
4307	Binary Exponent of Variable 2

Operational Boundaries (Note):

1. **Input Restriction:** During decimal data entry, the input base-10 exponent is strictly bounded to the range $-99 \leq E_{10} \leq +99$.
2. **Output Restriction:** During printing operations via the paper tape typewriter, the printed base-10 exponent is likewise bounded to $-99 \leq E_{10} \leq +99$.
3. **Binary Underflow/Overflow Guard:** When executing internal calculations or storing the contents of the virtual Floating-Point Accumulator into memory, the computer will *not* halt (i.e., no hardware overflow/underflow fault is triggered), provided that the internal binary exponent satisfies the following safety threshold:

$$-333 \leq E_2 \leq +333$$

4. PSEUDO-INSTRUCTIONS

The following section details the complete catalog of the 33 virtual instructions compiled for the "Floating-Point Interpreter System 2" and explains their operational mechanics. Any reference to the "Accumulator" designates the designated two-cell memory block allocated to the software Floating-Point Accumulator.

Arithmetic Instructions

Note: `XXXX` represents the absolute starting memory address of the target floating-point number's mantissa.

Opcode	Name	Operational Action / Mechanics
BXXXX	Bring	(XXXX) $\rightarrow$ Accumulator
AXXXX	Add	(Accumulator) + (XXXX) $\rightarrow$ Accumulator
SXXXX	Subtract	(Accumulator) $-$ (XXXX) $\rightarrow$ Accumulator
DXXXX	Divide	(Accumulator) / (XXXX) $\rightarrow$ Accumulator
PXXXX	Place	(XXXX) $\rightarrow$ M Register
MXXXX	Multiply	(M Register) $\times$ (XXXX) $\rightarrow$ Accumulator
NXXXX	Multiply Cumulatively	[(M Register) $\times$ (XXXX)] + (Accumulator) $\rightarrow$ Accumulator
XD000y	Divide by 2^y	(Accumulator) / 2^y $\rightarrow$ Accumulator $(0 \leq y \leq 9)$
XM000y	Multiply by 2^y	(Accumulator) $\times$ 2^y $\rightarrow$ Accumulator $(0 \leq y \leq 9)$
HXXXX	Hold	(Accumulator) $\rightarrow$ XXXX
CXXXX	Clear	(Accumulator) $\rightarrow$ XXXX; then $0 \rightarrow$ Accumulator

For the power-of-two scaling commands (XD000y and XM000y), the accumulator strictly maintains its internal normalized floating-point structure throughout the bit-shifting sequence.

Logical or Branch Instructions

Note: XXXX represents the target instruction address on the magnetic drum.

Opcode	Name	Operational Action / Mechanics
UXXXX	Unconditional Jump	Forces execution loop to target cell XXXX.[17]
TXXXX	Test Minus	Jumps to XXXX if (Accumulator) < 0; else executes next sequential lin
800TXXXX	Conditional Jump	Jumps to XXXX if (Accumulator) < 0 OR if console switch PST is ON.
XXXXX	Set Address	(Address Part of XXXX) $\rightarrow$ Address Register
IXXXX	Increment Address	(Address Register) + XXXX $\rightarrow$ Address Register[18]
YXXXX	Store Address	(Address Part of Address Register) $\rightarrow$ Address Part of cell XXXX
ZXXXX	Zero Test	(Address Register) $-$ XXXX. If result $= 0$, skip next instruction; else
RXXXX	Return	Links subroutines: (PC $+ 2$) $\rightarrow$ Address Part of target cell XXXX.

Auxiliary Instructions

Note: The address field (XXXX) for all auxiliary operations must be explicitly configured as 0000.

Opcode	Name	Operational Action / Mechanics
U0000	Swap Registers	(M Register) $\longleftrightarrow$ (Accumulator)
B0000	Set Plus Sign	Forces (Accumulator) positive [Absolute Value: $\lvert Accum$
T0000	Set Minus Sign	Forces (Accumulator) negative [Inverse Absolute Value: $-\lvert Accum$
Y0000	Change Sign	(Accumulator) $\times -1 \rightarrow$ Accumulator [Toggle si
Z0000	Operational Stop	Halts execution if console switch PS-16 is ON. Resumes on manual START.
E0000	Exit Interpreter	Safe exit from Virtual Machine; returns to native hardware control at nex

Input/Output Instructions

Note: The address field for all input/output commands must be set to 0000.

Opcode	Name	Operational Action / Mechanics
I0000	Data Input	Diverts to tape-reader module. Parses, normalizes, and stores decimal variable
P0000	Data Output	Prints/Punches the current Accumulator value. Accumulator data remains unalter

During tape data input (I0000), sequential tracking remains under tape control until a designated physical stop-code is pulled from the reader, passing execution control back to the next sequential interpreter instruction.

Function Instructions

Note: The address field for all transcendental operations must be set to 0000. The input argument must already sit in the Accumulator.

Opcode	Name	Operational Action / Mechanics
R0000	Square Root	$\sqrt{\text{(Accumulator)}} \rightarrow$ Accumulator. Negative inputs trigger an immediate erro
S0000	Sine	$\sin$ (Accumulator) $\rightarrow$ Accumulator. Input argument must be formatted in radi
Q0000	Cosine	$\cos$ (Accumulator) $\rightarrow$ Accumulator. Input argument must be formatted in rad
A0000	Arctangent	$\arctan$ (Accumulator) $\rightarrow$ Accumulator. Resulting output is scaled in radians
N0000	Natural Logarithm	$\ln$ (Accumulator) $\rightarrow$ Accumulator.
H0000	Exponential	$e^X \rightarrow$ Accumulator, where X represents the initial value held in the Accu

System Composition Architecture Note: The core arithmetic instructions, branch/jump vectors, address register manipulation routines, auxiliary commands, and the primary Square Root (R0000) engine form the permanent hardcoded baseline footprint of the interpreter core loop.

Conversely, the individual Data Input, Data Output, and specialized transcendental math packages were developed as independent software library modules. To maximize available user space on the drum, these external extensions only need to be loaded into memory if the user's application program explicitly calls them.

5. DATA INPUT

The pseudo-input instruction (I0000) requires a data paper tape formatted precisely as follows:

1. A starting address block.
2. The raw data formatted as signed decimal integers coupled with their respective exponents.

3. A designated group-termination code at the end of each distinct cluster of numbers.

4. A final master execution stop-code at the terminal end of the data tape.

The Starting Address

Every independent data group must begin with its target starting memory location, designated in decimal Track/Sector notation. This must be an absolute address and must be immediately followed by a physical stop-code (`'`).

Example: The entry `4200'` instructs the input subroutine to begin storing the incoming converted floating-point variables sequentially starting at memory cell `4200`.

The Data Structure

Following the starting address line, one or more signed decimal integers and their exponents are fed into the reader. The subroutine converts this decimal matrix into internal binary floating-point representations and stores them in consecutive, two-cell blocks on the drum.

Each individual floating-point value must be explicitly split into *two distinct words* on the paper tape, each line capped with a hardware stop-code (`'`).

- **Negative values:** The minus sign (`-`) must immediately precede the very first digit of the first word.
- **Positive values:** A space may be inserted before the first digit, but a physical plus sign (`+`) is strictly prohibited.

The structural payload layout across the tape lines is split as follows:

- **Word 1:** Contains the algebraic sign bit and the first seven digits of the decimal integer.
- **Word 2:** Contains the final two digits of the decimal integer, followed immediately by the sign and the two digits of the base-10 exponent.

Formally, the physical tape notation maps to this exact structural syntax:

$$\begin{aligned} &\textbf{Line 1 (Word 1):} \quad \pm d_1 d_2 d_3 d_4 d_5 d_6 d'_7 \\ &\textbf{Line 2 (Word 2):} \quad d_8 d_9 \pm e_1 e'_2 \end{aligned}$$

Where $\pm d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8 d_9$ constitutes the total 9-digit signed decimal integer, and $\pm e_1 e_2$ represents the 2-digit decimal scaling exponent.

The mathematical value of the processed data variable is determined by the underlying relationship:

$$\text{Represented Value} = \text{Integer} \times 10^{E_{10}}$$

Where the exponent boundaries are hardcoded at: $-99 \leq E_{10} \leq +99$.

Input Encoding Examples

- **Example 1:** Consider the fraction 0.0900000521374209. This corresponds mathematically to the scientific notation expression: $521374209 \times 10^{-16}$. To feed this value into the interpreter, it must be punched onto the paper tape as two distinct lines:

```
5213742'
09-16'
```

- **Example 2:** Consider the large whole number 5522829640000000. This corresponds to the mathematical notation: $552282964 \times 10^{+07}$. It must be presented to the reader as:

```
5522829'
64+07'
```

The Termination Stop-Codes

The system defines two distinct classes of termination stop-codes. Their behavior varies fundamentally based on their context and placement on the data coding sheet:

1. **Group Termination Code:** Signifies the end of the current numerical data block. It commands the system to immediately begin reading the next subsequent address and data group on the tape.
2. **Master Input Stop-Code:** Signifies the total completion of data entry. It triggers an automatic terminal carriage return, shuts down the tape reader pipeline, and gracefully jumps execution control back to the next sequential line of the user's main program.

7 Number Conversions

7.1 Decimal/Hexidecimal Conversion

8 Program input

8.1 Program Loader 1

J1-10.0

PURPOSE

Program Input 1, J1-10.0, [Translator's note: this program is known more colloquially as "PIR".] is a memory program. It facilitates the reading of hexadecimal tapes—whether arbitrarily relocated or not—and calculates a checksum during the reading process. It can also read and check arbitrarily relocated decimal programs.

Program J1-10.0 performs the following functions:

1. Reading, encoding, and storing instruction words to predefined memory locations.
2. Reading and storing hexadecimal or alphanumeric words to predefined memory locations without encoding.
3. Setting an address modifier in a program to a specific value.
4. Reading decimal addresses incremented by the modifier value.
5. Setting the "Start Fill" counter to a specific address value.
6. Reading specific instructions and executing them immediately.
7. Reading decimal addresses, followed by a jump either to this memory location or to the memory location specified by the address plus modifier.
8. Reading a decimal address; the contents of this memory location appear in the accumulator.
9. Reading, encoding, and storing arbitrarily relocatable decimal tapes.
10. Reading and storing hexadecimal tapes.
11. Selecting a specific input unit.

Hexadecimal tapes are usually generated by a hexadecimal dump program, such as the J4-10.0 program.

Memory usage

Hexadecimal input only	2 tracks
Combined input	3 tracks, 29 sectors
Buffer usage	None

textsf

- `MNNKHHHH`

  This keyword reads `NN` hexadecimal words in ascending order into consecutive memory locations, starting at memory location `HHHH` plus the modifier. The `K` is a store instruction. The input keyword contains a flag bit (bit 31); each of the NN words that are to be modified must also contain a flag bit.

  The address modifier (see below) is added to the operand address portion of each flagged word before it is stored.

  A checksum is calculated from all `NN` words as they are being stored plus the input keyword (all values are checksummed without the address modifier). If the calculated checksum does not match the checksum on the tape following these words, an error stop occurs. If the checksums match, the computer requests a new input keyword.

  Example: Let the modifier be 1200 (decimal) and the input keyword be M40K1201' (hexadecimal). The next 64 (40 hex) hexadecimal words are read and modified (if they have a flag bit) and are stored consecutively, beginning in memory location 3000.

- `VNNCHHHH` -

  The hexadecimal fill keyword for non-relocatable [absolute] tapes reads in `NN` hexadecimal words starting with memory location `HHHH`. `C` is a store instruction. The input keyword does not contain a flag bit. The most recently-entered address modifier value is added to the operand address portion of all hexadecimal fill words that have a flag bit as they are stored.

  A checksum is calculated from all `NN` words as they are being stored plus the input keyword (all values are checksummed without the address modifier). If the calculated checksum does not match the checksum on the tape following these words, an error stop occurs. If the checksums match, the computer requests a new input keyword.

- `VNNCTTSS`

  Example: The input keyword `V40C1100'` causes the following 64 (40 hex) hexadecimal words to be stored, beginning in memory location 1700.

  When reading hexadecimal tapes with `V` or `M` keywords, the Program Input Routine verifies each stored word; specifically, after a word is stored, the value in memory is compared with the read-in value. If the two values do not match, an error stop occurs.

- `/000TTSS`

  The Modifier Input Keyword sets the address modifier to the address `TTSS` contained within the input keyword.

  When entering hexadecimal tapes, this Address Modifier is added to all keywords and words that contain a flag bit.

When entering decimal instruction words, the modifier is added to the operand addresses of all instructions except those preceded by an "X".

The modifier remains unchanged until it is replaced by a new Modifier Input Keyword.

Example: The input keyword `/0001200'` causes 1200 to be added to all flagged operand addresses and keywords.

- `.000TTSS'`

  The Absolute Stop and Jump Input Keyword causes the computer to stop. If the Start button is subsequently pressed, a jump occurs to cell `TTSS`. If the PS-32 switch is set to ON, execution continues at the target address.

  Example: The input keyword .0001663′ would cause the computer to jump to 1663 and halt execution (if PS-32 is off).

- `.100TTSS'`

  The Relative Stop and Jump Input Keyword causes the computer to stop. If the Start button is subsequently pressed, a jump occurs to cell `TTSS` plus the modifier. If the PS-32 switch is set to ON, the stop is suppressed.

  Example: If the modifier has been set to 1200, then the input keyword `.1001663'` would cause a jump to 2863.

- `S000TTyy'`

  The Selection Keyword selects the specific input unit designated by the decimal track address `TT`. (`yy` can be any arbitrary sector address; it is ignored, but must be supplied). A list specifying the corresponding input units for specific track addresses can be found in the LGP-21 Programming Manual.

  After entering the Selection Keyword, the computer stops. After pressing the Start button, the computer continues reading via the selected input unit.

  Restarting J1-10.0 by jumping to the start of the program will automatically reselect the primary Flexowriter for input. Reading will then continue via the Flexowriter until a new Selection Keyword selects a different unit.

  Example: If 32 refers to a second Flexowriter, then the keyword `S0003200'` selects this second typewriter for further input.

Decimal input

- `000TTSS'`

  The Decimal Fill keyword causes the input program to store decimal words in ascending order into consecutive cells, beginning with memory location `TTSS`.

Any valid address followed by a stop code may be used. Every decimal input should be preceded by a Decimal Fill keyword. The "start fill" address is stored in the start-fill counter. This memory location of the subroutine contains the address for the decimal input. The counter is incremented by 1 each time after a word (not a keyword) is read and stored. The read-in words are stored consecutively until a new decimal fill keyword is read or the input is terminated.

Example: The fill keyword `;0000400'` would store read-in words beginning with memory location 0400.

- `,00000NN'`

  The keyword for entering hexadecimal words causes `NN` hexadecimal words to be stored, specifically beginning in the memory cell whose address is held in the start-fill counter. `NN` must be specified in decimal and can take a value from 1 to 63. The stored words are not incremented by the address modifier. Leading zeros may be omitted for the hexadecimal words that are to be read in.

  Each hexadecimal word and keyword must be followed by a stop code. If a zero is to be entered, it is not necessary to write a `0`. A stop code alone is sufficient to indicate a zero. Each hexadecimal word must not consist of more than 8 characters.

  Example: The input keyword `,0000003'` followed by the three hexadecimal words `-2'JK73''` would cause the values `00000002`, `0000JK73`, and `00000000` to be stored into the next three consecutive cells.

- `A00000NN'`

  The keyword for entering alphanumeric words causes `NN` alphanumeric words to be stored, beginning in the memory cell whose address is held in the start-fill counter. The alphanumeric words are shifted two places to the left and stored without binary conversion. `NN` must be specified in decimal and can take a value from 1 to 63.

  Each word can contain up to five characters in 6-bit representation (if more than five characters per word are entered, only the last five are retained). The last stored word receives a group-end indicator, i.e., 1@30. All characters except tape feed (blank tape), delete code, and stop code can be entered; for example, lowercase letters can be stored.

  Example: The input keyword `A0000004'` followed by the four words `TOTAL' UNIT'S PRO'DUCED'` would cause the information `TOTAL UNITS PRODUCED` to be stored into four consecutive memory locations without conversion into binary.

- `+00LTTSS'`

  The Instruction Input Keyword is treated by the subroutine exactly like an instruction.

The arbitrary instruction `LTTSS` contained within the keyword—where `L` is any positive machine instruction and `TTSS` represents any valid address—is executed immediately after either the input of a subsequent hexadecimal word or the input of a stop code (if no subsequent word is required). The address portion of the input keyword is not altered by the Address Modifier.

Each hexadecimal keyword and input word must be followed by a stop code. Left-side leading zeros within a word may be omitted. The computer returns to an input command without the execution of the instruction causing a branch. Jump instructions should be avoided.

Note: The instruction contained within the input keyword does not alter the fill counter.

Example: The input keyword `+00h1637'` followed by the hexadecimal word `73W08'` would cause the value `00073W08` to be stored in memory cell 1637.

- `-000TTSS'`

  The Sequential Display Input Keyword causes the contents of memory cell `TTSS` to appear in the accumulator when the computer stops. A start signal causes a return to an input command for a new keyword, provided that the `PST` switch is off. If the `PST` switch is on, the contents of `TTSS`$+i$ (where $i = 1, 2, \dots$) will appear in the accumulator.

  With each start signal, i is incremented by 1. It is also possible to obtain the address of the displayed word by switching to single-step operation and starting the machine. (The `B` instruction that fetched the information just displayed will appear in the accumulator.) `TTSS` may be any arbitrary valid address.

  Example: The input keyword `-0003263'` would directly display the contents of memory cell 3263 in the accumulator.

  Every entered word whose leftmost four bits correspond either to an "8" (negative instruction) or a zero (positive instruction) is treated as an instruction. The operand address is altered by the Address Modifier, unless the instruction is preceded by an "X". The subroutine allows the inclusion of index tags (used by some interpretive systems) immediately preceding the instructions. The inclusion of an "X" or an index tag does not alter the instruction in memory. If the leftmost four bits of a word correspond to any of the numbers 9 through 13, an error stop occurs (see the list of error stops).

  Example: Let the Address Modifier be 0600. Different types of instruction words would be stored as follows:

Input	Storage
B 1234	B 1834
XB 1234	B 1234
800T 1234	800T 1834
80XT 1234	800T 1234
8B 1234	8B 1834
X8B 1234	8B 1234

The subroutine consists of two parts: hexadecimal input and decimal input. The hexadecimal input section can be used independently of the decimal input; however, the decimal input is dependent on the hexadecimal section, which must always be stored in memory.

CALLING SEQUENCE

The program is called manually by jumping to the starting address (regardless of which section is being used).

OPERATING INSTRUCTIONS

PROGRAM INPUT The tape contains the program in relocatable hexadecimal form with its own bootstrap loader. Operation is performed as follows:

a) Place the program tape into the typewriter's reader mechanism and release all levers.

b) Set the mode selector switch to `Manual Input` on the console.

c) Press `Start Read` on the Flexowriter.

d) Press `Fill Clear` on the console.

e) Press `Start Read`.

f) If the program is to be relocated, enter the starting address in hexadecimal form via the typewriter (typically the starting address is 0000); otherwise, omit this step.

g) Set the mode selector switch to `Single Step`.

h) Press `Execute` on the console.

i) Repeat steps b) through h) until "normal" is completely printed out by the computer.

j) Set the mode selector switch to `NORMAL`.

k) Press `START` on the console.

l) The program automatically continues reading in.

Note: If the PS-32 switch is turned *off*, the computer stops after reading in the hexadecimal input section. If the PS-32 switch is turned *on* or if START is pressed, the decimal input section or a hexadecimal object program tape will be read in.

PS-32 should be turned *off* before the decimal input section is completely read in; otherwise, the reader mechanism will continue to operate without tape after the read-in process is complete.

The bootstrap always occupies memory cells 0000 and 0002 . . . 0008 (absolute), while the remainder resides in the area designated for the hexadecimal input section.

INSTRUCTIONS FOR RELOADING

The bootstrap remains intact in memory if the program has not been relocated. To re-load the program without the bootstrap, proceed as follows:

a) Place the program tape into the reader, specifically aligning it at the blank tape section between the bootstrap and the program.

b) Set the mode selector switch to Manual Input on the console.

c) Enter C0000 via the typewriter.

d) Press Fill Clear on the console.

e) Enter the modifier in hexadecimal form (all 8 characters), or enter 8 zeros if the program is not to be relocated.

f) Set the mode selector switch to Single Step on the console.

g) Press Execute on the console.

h) Set the mode selector switch to Manual Input on the console.

i) Enter U0008.

j) Press Fill Clear on the console.

k) Set the mode selector switch to Single Step on the console.

l) Press Execute.

m) Set the mode selector switch to NORMAL on the console.

n) Press Start. The program will be read in automatically.

The C0000 and U0008 Instructions

In the LGP-21 instruction set, C is the Clear and Transfer instruction (storing the accumulator contents in memory), and U is the Unconditional Jump instruction. Entering U0008 manually forces the to skip the initial bootstrap block and jump straight to the start of the primary loader routine at location 0008.

REQUIRED INPUT/OUTPUT UNITS

Input units other than the standard typewriter must be specified by a selection code. Output units are not required.

8.1.1 THE EFFECT OF THE PROGRAM JUMP SWITCHES

- `PS-32` (Program Switch 32)
  - `OFF`:
    - The computer stops before executing a jump when a Stop and Jump Input Keyword is used.
    - The computer stops after reading in the hexadecimal input section of the subroutine.
  - `ON`
    - No stop occurs when a Stop and Jump Input Keyword is used.
    - No stop occurs after reading in the hexadecimal or decimal section of the subroutine.
- `PST` (Program Switch-Typewriter)
  - `OFF`: Only the contents of a single, specific memory cell will appear in the accumulator.
  - `ON`: Consecutive memory cells can be loaded into the accumulator by repeatedly pressing the Start button.

MESSAGES

E (Input Error)

Cause:

- Hexadecimal Tape Input: This character indicates a checksum error.
- Decimal Tape Input: This character indicates an invalid input code.

Remedy:

a) Move the tape back to the last M or V input keyword.

b) Press START (the input unit does not need to be re-selected).

Remedy for an Invalid Input Code (the last word read contains the error):

a) Press Manual Input on the typewriter.

b) Set the mode selector switch to Manual Input.

c) Enter a jump to the starting address of the subroutine in hexadecimal form (i.e., U0000 or the respective starting address). If the subroutine begins in memory location 0000, steps c), e), and f) may be omitted.

d) Press Clear and Input (Fill Clear) (TR).

e) Set the mode selector switch to Single Operation (Single Step) (TR).

f) Press Execute (TR).

g) Set the mode selector switch to Normal (TR).

h) Press Start (TR). The typewriter lamp lights up.

i) Input the correct input code.

j) If the typewriter is the active input unit, release the Manual Input lever, and the tape will continue reading.

k) If another input unit is used, enter its selection code, press the Computer Start lever, then press Start (TR), and the tape will continue reading.

W (Write/Verify Error)

This printout indicates that a verification error occurred during the reading of a hexadecimal tape (i.e., the word actually stored in memory did not match the word read from the tape).

This error is remedied in the same manner as a checksum error.

Tape verification

By modifying two instructions in the subroutine, it can be used to verify hexadecimal tapes. For example, after a hexadecimal tape has been punched, alter the two instructions in the subroutine and then read the hexadecimal tape in.

The subroutine will read the tape (without storing it) and compare it word-for-word with memory. The checksums are also verified. If the comparison is unsuccessful, the program halts with the error stop "W" or "E". The tape is correct if it reads through without difficulty.

The instructions to be modified are:

Memory Location	Old Contents	New Contents
L + 24	H J	U 0025
L + 31	Y 0024	U 0032

Address Notation Legend

`Ø` Machine instruction code, represented as a letter.

`TT` Track portion of the address, in decimal (00–63).

`SS` Sector portion of the address, in decimal (00–63).

`NN` Number of words to read, in decimal.

`HHHH` Hexadecimal memory address.

Instruction Keyword Quick Reference

Keyword Format	Basic Meaning
ØTTSS	Instruction – may be modified.
XØTTSS	Instruction – may not be modified.
8ØØTTSS	Negative instruction – may be modified.
8ØXØTTSS	Negative instruction – may not be modified.
IØTTSS	Instruction with index tag – may be modified.
XIØTTSS	Instruction with index tag – may not be modified.

Detailed Operation Summary

Keyword	Operation & Detailed Description
MNNKHHHH	*Fill with Relocatable Words* Read NN hexadecimal words into memory, starting at location HHHH + modifier. If bit 31 is 1, operand address is modified. Checksum calculated and compared with word NN+1.
VNNCHHHH	*Fill with Non-relocatable Words* Read NN hexadecimal words into memory, starting at location HHHH. If bit 31 is 1, operand address is modified. Checksum calculated and compared with word following the NN words previously read.
/000TTSS	*Set Modifier* Sets an address modifier to TTSS. Modifier is added to all hexadecimal fields that have bit 31 set, and to the addresses of decimal instructions not preceded by an X.
.000TTSS	*Stop and Jump Absolute* Sets program counter to TTSS and stops execution. If PS-32 is ON, execution continues.
.100TTSS	*Stop and Jump Relative* Sets program counter to TTSS + modifier and stops. If PS-32 is ON, execution continues.
S000TTxx	*Device Select* Selects (decimal) input device TT (xx is ignored) and stops. To continue, press START.
;000TTSS	Decimal Fill Reads decimal instructions into memory starting at address TTSS.
,00000NN	Hexadecimal Fill Reads NN hexadecimal words into the following NN consecutive memory cells.

Keyword	Operation & Detailed Description
A00000NN	Alphanumeric Fill Reads NN alphanumeric words into the following NN consecutive memory cells.
+00ØTTSS	Execute Immediate Executes the instruction ØTTSS contained within the keyword. If the instruction has no operand, enter a stop code. Otherwise enter the operand word in hexadecimal notation an a stop code.
-000TTSS	Sequential Dump The contents of memory cell TTSS are shown in the console's accumulator display when the computer stops. After pressing START, the computer requests an command if PST is OFF. If PST is ON, pressing START causes the contents of TTSS+1 to appear in the accumulator, and so on. The address of the cell currently shown in the accumulator display appears in the accumulator if you switch to "Single Operation" and press Start.

8.2 Program Loader 2

8.2.1 PURPOSE

Program J1-10.1 is an interactive programiming monitor. Similar to J1-10.0, it accepts relocatable decimal programs, hexadecimal words, and alphanumeric words. It also reads non-relocatable hexadecimal programs, calculates checksums, and compares them with the checksums on the hexadecimal tape. Furthermore, this program can be used to load memory contents into the accumulator; commands can be executed directly without being stored; and it allows for manual branching (jumping).

Memory usage

Program storage	3 tracks
Buffer usage	None

The inputted pieces of information are keywords that indicate to the program: a) the format of the words that follow, b) the words that are to be stored, c) the words that are to be executed immediately.

These input keywords, and the formats of the words that follow them, are discussed on the following pages. (Please note that for each stop code (') specified in the description, START COMP [Start Computer] must be pressed if the input is being entered via the typewriter keyboard.)

;000TTSS' Fill Keyword (Start Fill) The fill keyword indicates the starting address TTSS for a series of information to be stored. This value is kept in the "fill counter" —- a memory cell within the subroutine—and indicates into which cell the next word is to be stored. The counter is incremented every time a word (not a keyword) is read and stored.

The words to be read in can be in instruction form, as hexadecimal words, or as alphanumeric words. The words are placed into consecutive memory cells until inputting is finished or until another fill keyword appears. The start-fill address must be in decimal track/sector notation, followed by a stop code. Any real address can be used. For example, the keyword ;0001600' causes the subsequent information to be stored in consecutive cells, beginning in 1600.

ØTTSS' Instruction The subroutine recognizes an instruction by zeros in bit positions 0 to 3, or in the case of negative instructions, by a "1" in the sign position and zeros in bit positions 1 to 3. The instruction consists of the alphabetical command symbol Ø, followed by the operand address TTSS.

If the accumulator is cleared prior to input, leading zeros (those preceding the instruction) may be omitted. The entire instruction is converted into its binary equivalent and stored in the cell specified by the fill counter.

[translator notes: Technical Context and LGP-21 Word Structure This passage gives an excellent look into the internal architecture and machine word layout of the LGP-21:

Bit Positions 0–3: In a standard 32-bit word on the LGP-21, the highest-order bits are used to determine the type of word being parsed. By checking

for zeros here, the monitor dynamically distinguishes a raw machine instruction from a data word or a keyword macro without needing explicit prefixes.

Sign Bit ("1" in the sign position): Negative instructions (often used for specific accumulator logic or inverted arithmetic operations) utilize the standard sign bit position while keeping the rest of the operational identifier bits clean so the parser can still route the address correctly.]

/000TTSS' Modifier The address portion (TTSS) of this keyword is stored within the subroutine in the cell designated for the modifier.

The modifier (i.e., the contents of this cell) is added to the operand address of all sequentially read instruction words, with the exception of instructions preceded by an "X".

The modifier remains effective until it is replaced by another modifier keyword. Any real address in decimal track/sector notation can be used and must be followed by a stop code.

For example, the keyword /0001100' stores the value 1100 in the modifier cell. 1100 is then added to the address of all subsequent instructions that are not preceded by an "X":

Instructions Read Instructions Stored B2611' B3711 XB2611' B2611 800Z0400' 800Z1500 80XZ0400' 800Z0400

.000TTSS' Stop and Jump

This keyword is executed in two steps. First, the computer halts. Then, after pressing the START button once, a jump to TTSS occurs. Any real address in decimal track/sector notation can be used. Every keyword must be followed by a stop code. If PS-32 is pressed (turned ON) before the keyword is entered, the halt is suppressed, and execution continues after the jump.

For example, the keyword .0002000' causes the computer to halt; after a start signal, a jump to 2000 occurs and execution continues from there.

+000TTSS' Direct Command (Direktbefehl) This keyword enables the programmer to have the computer execute an instruction directly. The instruction ØTTSS contained within the keyword (where Ø represents a command symbol and TTSS is any real address in decimal track/sector notation) is executed following the entry of a hexadecimal word and/or a stop code. That is, it executes either following the entry of the hexadecimal word to which the keyword refers, or simply following the entry of a stop code if no hexadecimal word is required.

The address portion of the command keyword is not modified; likewise, any hexadecimal word entered afterward is not modified. Every keyword and every hexadecimal word must be followed by a stop code. Leading zeros in the hexadecimal word may be omitted.

[translator notes: This command highlights just how powerful J1-10.1 is as an interactive machine monitor. The +000TTSS' keyword essentially acts as a dynamic "execute single instruction" vector:

Immediate Execution Without Drum Overhead: You can force the CPU to execute a command right out of the input buffer without having to write it to a track on the drum first.

Hexadecimal Word Injection: If the instruction requires an operand value in the accumulator to work with (such as an add, load, or store instruction), you

can type a raw hex data word right after the keyword. The monitor streams that data into the accumulator first, and then immediately fires the command.

Bypassing Modification: Because this is intended for manual console debugging and testing, the routine safely bypasses the standard relocation modifier register. This ensures that what you type is exactly what gets executed, with no unexpected address translation.]

For example, the direct command +00H1637' followed by the hexadecimal word 73W08' causes the value 00073W08 to be stored in cell 1637; the word +00U1735" causes a jump to 1735.

Note The keyword for the direct command does not alter the fill counter.

[translator notes: This is a phenomenal look at the classic LGP-21 / LGP-30 machine language syntax, specifically regarding its opcodes and the unique hexadecimal character set:

H for Hold and Store: In the first example, H is the command symbol for "Hold and Store" (storing the contents of the accumulator into a memory location without clearing the accumulator). You poke the hex data word 73W08' into the typewriter, it loads into the accumulator, and the direct instruction H1637 immediately dumps it into memory address 1637.

U for Unconditional Transfer: In the second example, U is the command symbol for an "Unconditional Transfer" (a standard jump instruction). Notice the trailing double stop code (") in the manual text: +00U1735". The first tick mark closes out the keyword word entry, and the second tick mark serves as the trailing stop code executing the immediate action since no secondary hexadecimal data word was required.

The Letter W as a Hex Digit: The hexadecimal word 73W08 contains the letter W. For readers unfamiliar with the LGP architecture, it used a non-standard, typewriter-friendly hex character set where the letters f, g, j, k, q, w represented the decimal values 10 through 15 (A through F). The character W explicitly represents the value 15 (or F in modern hex notation).]

,00000NN' Hexadecimal Words This keyword causes the following NN words to be stored as hexadecimal words in consecutive memory cells according to the current state of the fill counter. NN must be less than 64. The words to be stored must be in hexadecimal format and are not modified.

For example, ;0002200',0000003',3W7'140Q" causes the memory to look like this after entry:

2200 000003W7 2201 0000140Q 2202 00000000

The Format of Hexadecimal Words: A hexadecimal word may contain a maximum of eight characters. Any combination of the characters 0,1,...,9 and F,G,J,K,Q,W may be used; zero is the smallest value and WWWWWWWW is the largest value the computer can accept. If a word is zero, only a stop code is required. Leading zeros may be omitted; every word must be followed by a stop code.

[translator notes: The example string features three consecutive inputs after setting the fill address to 2200: 3W7', 140Q', and a final empty input before a stop code '. Because leading zeros can be dropped and a solo stop code represents

zero, the monitor correctly shifts and pads them into full 32-bit words, leaving cell 2202 cleared out as all zeros.]

VNNCHHHH' Hexadecimal FillThis keyword causes the following NN hexadecimal words to be stored in consecutive cells, beginning with HHHH (a fill keyword is not required). The C is a clear command. The keyword is in hexadecimal notation and is normally provided by a hexadecimal output program, where it stands at the beginning of a block of 64 or fewer hexadecimal words.During input, a checksum is computed from the keyword and all hexadecimal words. Once all NN words are stored and if PST is off, this checksum is compared with the tape checksum at the end of the corresponding block. If they are equal, reading continues; if they are unequal, an error stop occurs. See under "Error Stops." If PST is switched on, the checksum comparison is skipped.For example, the keyword V1WC2W00' causes the next $1W_{16} = 31_{10}$ hexadecimal words to be stored, beginning in cell $2W00_{16} = 4700_{10}$.

[translator notes: The "C" Clear Command: The manual notes that `C` acts as a clear command within the macro. In the context of automated tape loading, this likely instructed the internal parser to clear the checksum accumulator or reset specific input buffers before dumping the next payload block into memory.

Automated Block Loading: This V keyword represents the automated counterpart to the manual keywords you looked at earlier. Instead of an operator typing line-by-line, an output program (like a paper tape punch utility) would prepend blocks with this header. The inclusion of an automatic checksum validation loop (which could be bypassed via the physical PST switch on the console) proves this was the primary mechanism for streaming large software binaries reliably.]

A00000NN' Alphanumeric WordsThis keyword permits the entry of alphanumeric information in a 6-bit format. Information read in this manner can be punched back out directly by print commands without any conversion, or handled by an alphanumeric print program.Every word, with the exception of the last word of a block, must contain exactly 5 characters (the last word may contain fewer) and must be followed by a stop code. The quantity NN of words to be stored must be less than 64. The NN words are stored in consecutive cells (at $q = 29$), starting at the cell indicated by the current state of the fill counter.For example, the entry A0000005' DEPRE'SS ST'ART T'O CON'TINUE' causes the string "DEPRESS START TO CONTINUE" to be stored across five consecutive cells.

[notes: 6-Bit Character Packing: This section reveals exactly how text string data is packed into the LGP-21's 32-bit architecture. Because characters are encoded in a 6-bit format, a single 32-bit machine word can hold a maximum of 5 characters ($6 \times 5 = 30$ bits), leaving 2 bits leftover.The Fixed Scaling Factor ($q = 29$): The mention of q=29 refers to the binary point placement (or scaling factor) within the 32-bit register. In LGP programming, specifying the q fraction bit position ensures that the packed 6-bit character stream aligns perfectly with the internal shift registers used by alphanumeric input/output subroutines.The Disappearing Spaces in the Example: If you count the characters in the example string DEPRE'SS ST'ART T'O CON'TINUE', you can see

exactly how the 5-character word limit maps out when trailing stop codes act as delimiters:Word 1: DEPRE (5 chars)Word 2: SS ST (5 chars, including the space)Word 3: ART T (5 chars, including the space)Word 4: O CON (5 chars, including the space)Word 5: TINUE (5 chars)Notice that the original German manual translated the English word "DEPRESS" into an exact 5-character parsing structure for the example, highlighting that even spaces count toward the strict 5-character allocation per memory cell.]

-000TTSS' Inquiry Keyword (Frageschlüsselwort) The inquiry keyword brings words from memory into the accumulator. If the computer is equipped with an oscilloscope, the programmer can view the words without any instructions being executed. Only the word located in TTSS is brought in at any one time. Each time START is pressed, the next word in ascending sequence appears. The following operation must be performed to return to normal operation[19]:

1. Set the selector switch to Manual Input.
2. Press Enter and Clear.
3. Set the selector switch to Normal.
4. Press START.

For example, the keyword `-0002003'` brings the contents of location 2003 into the accumulator. After pressing START, the contents of 2004 appear, and so on. This process can be continued until the above operation interrupts the sequence.

S000TTss' Input Device Selection This keyword causes the selection of an alternative input device (other than the default Flexowriter). The program reverts to Flexowriter input at the start (with a branch to 0000) and following error stops. The selection codes and their corresponding units are listed in the *LGP-21 Programming Manual*[9].

The selection codes must be specified in decimal track/sector notation `TTSS`, followed by a stop code (the sector portion value is irrelevant). For example, if another input device could be selected by the identification number 32, the selection keyword `S0003200'` causes this device to be selected for this routine's input.

Tapes to be read by the program should be capable of being typed out on the typewriter; that is, a carriage return should occur after every four to eight words[20].

[19] **Escaping the Hardware Loop**: Because this monitor instruction takes over the machine's primary execution cycle to step through memory addresses sequentially with each physical button press, it puts the hardware into a state that cannot be broken by software input alone. The explicit 4-step sequence forces a hard manual reset of the input buffer via the console switches to restore standard program control.

[20] Data tapes should contain a carriage return (CR) after every four to eight words simply because the mechanical realities of working with a hardware Flexowriter as console. If a continuous stream of data was read off a tape without embedded carriage returns, the physical typing element would reach the end of its travel at the right margin and continute to push against the margin as more characters are printed. This both imposes mechanical strain and overprints the input as an unreadable blur of characters at the edge of the log sheet.

If incorrect information is read, the program selects the typewriter[21], prints `err`, and halts. See "Error Stops".**??**

PROGRAM EXECUTION

The following steps are necessary to load the program from tape:

1. Press the MANUAL INPUT lever on the Flexowriter.
2. Set the selector switch on the system console to MANUAL INPUT.
3. Press FILL AND CLEAR on the console.
4. Set the selector switch to NORMAL.
5. Press the START button.

[notes: Why both MANUAL INPUT switches? This dual mechanical lockout prevented the computer from attempting to read from the console until the operator had explicitly prepared both the peripheral device and the central processing unit to accept manual keypresses.

Why the FILL AND CLEAR? Pressing FILL AND CLEAR clears the machine's primary instruction and input registers, ensuring that when the mode switch is set to NORMAL and START is pressed, the processor begins execution from the starting address of the `J1-10.1` routine without processing residual data or garbage instructions.]

PROGRAM OPERATION

1. The program tape contains the program in two versions: a) arbitrarily relocatable hexadecimal with its own bootstrap, and b) decimal in coding sheet format.

The input procedure is as follows:

a) Insert the tape into the typewriter reader. Ensure that all levers on the typewriter are released.
b) Press the I/O (E/A) button on the computer.
c) Set the selector switch to Manual Input.
d) Press Read Start on the typewriter.
e) Press Fill and Clear on the computer.
f) Press Read Start on the typewriter.
g) Set the selector switch to Single Operation on the computer.
h) Press Execute on the computer.
i) Repeat steps c through h twice; then proceed to j.
j) Set the selector switch to Manual Input on the computer.

[21]The fact that `J1-10.1` automatically reverts control back to the primary console ("standard typewriter") upon entry at 0000 or after an error stop is a fail-safe. If an external high-speed paper tape reader or secondary device feeds malformed data or fails mid-transfer, the system won't lock up or isolate the operator; it immediately brings the main console printer back online so the engineer can see the error state and intervene.

k) Press Read Start on the typewriter.

l) Press Fill and Clear on the computer.

m) Set the selector switch to Single Operation on the computer.

n) Press Execute on the computer.

o) Set the selector switch to Normal on the computer.

p) Press Start on the computer. The remainder of the tape is read in, and the computer halts.

q) Press Manual Input on the typewriter.

r) Press Start on the computer.

s) The typewriter's control lamp lights up, indicating that the computer is ready for input.

2. Rereading the Tape (Wiederholung des Einlesens) If the loaded program fails to work after step p, or if it was corrupted in memory after being read in, the bootstrap loader may still be intact in memory. In this case, the program can be re-read without going through the bootstrap input procedure as follows:

a) Place the tape in the reader, positioning it past the bootstrap section so that the bootstrap is not read again (tape run-out/leader).

b) Set the selector switch to Manual Input on the computer.

c) Press Read Start on the typewriter.

d) Press Fill and Clear on the computer.

e) Set the selector switch to Single Operation on the computer.

f) Press Execute on the computer.

g) Set the selector switch to Normal.

h) Press textsfStart on the computer. The tape is read in, and the computer halts. If the tape fails to read in, repeat the entire bootstrap procedure.

[notes: The Fragility of Hardware Bootstrapping: The convoluted routine in section 1 (toggling multiple times between Manual Input, Single Step, Fill and Clear, and Execute) perfectly illustrates the sheer mechanical effort required to manually clock the initial bootstrap instructions into an empty drum memory machine. This complex sequence was essentially a "hardware dance" designed to fill the instruction register with just enough code to let the paper tape reader take over automatically.

The "Relocatable Hex" vs. "Decimal Coding" Trick: The reference to the tape containing both a relocatable hexadecimal version and a decimal coding sheet version suggests a dual-purpose distribution. The relocatable hex version was meant for fast, automated loading via the monitor, while the decimal layout was likely structure-compatible with standard program printouts or direct memory-dump debugging workflows.

The Intact Loader Shortcut: Section 2 reveals a great workflow shortcut for operators. Because early magnetic drum memory retained its state unless explicitly overwritten or cleared, an input failure or program crash didn't necessarily wipe out the newly loaded bootstrap routine. Skipping the hardware-toggle sequence and positioning the tape directly past the loader code saved operators several minutes of tedious switch-flipping.]

3. Required Input/Output Units (Erforderliche Eingabe-/Ausgabeeinheiten) If an input unit other than the paper tape typewriter is to be selected, the

selection keyword (S) must be used. The subroutine selects the required output units automatically.

4. Meaning of the Program Switch Keys Switch State Function PS-32 Off The computer halts before jumping when a `Stop and Jump` keyword is executed. On The computer executes the `Stop and Jump` without halting. PST Off When inputting hexadecimal tapes, the checksums are compared. On When inputting hexadecimal tapes, the checksums are not compared. 5. Error Stops

Printout Meaning and Resolution err The computer has detected an error. After entering a hexadecimal fill keyword (V): a) Back up the tape to the last input keyword. b) Press START (on the computer). In the case of an incorrect or missing word: a) Press Manual Input (on the typewriter). b) Press START (on the computer). c) Enter the correct word. d) Release Manual Input (on the typewriter). e) Press START (on the computer). (Note: The last word read contains the error).

Note: Following an error stop, the subroutine automatically selects the typewriter for input.

[notes: Program Switch Names: The manual references two physical toggle switches on the LGP console:

PS-32: This corresponds to Program Switch 32 (often labeled as Transfer Switch 32 on some LGP hardware variations), which allows an operator to choose whether a programmed halt behaves as a hard pause for inspection or a transparent pass-through jump.

PST: This corresponds to the Program Stop/Test switch. Turning it On bypasses automated checksum checking entirely, a useful hardware override if a paper tape was slightly nicked or known to have a legacy block checksum error, but the operator wanted to force the machine to load it anyway.

The err State Recovery: The instructions reinforce that J1-10.1 is an interactive, conversational utility. If a tape misreads, typing `err` isn't a fatal crash. The recovery steps outline a clean process to switch back into manual control, type in a fixed data word to overwrite the glitchy paper tape frame, lock the typewriter's reader back down, and immediately resume tape execution.]

REMARKS

The subroutine cannot process data words (or constants) in decimal notation.

When a program is read in, the sign and the three most significant bits of each input word are examined, and then a jump is executed to the corresponding interpreting section of the subroutine.

The coding of a program can be broken down into logically independent groups, some of which can also be used in other programs; examples of this include all types of subroutines, standard input and output programs, and programs for mathematical sub-problems. In order for these instruction groups to remain permanently available, they should be assigned to memory areas where they can stay (so that two different blocks are not read into the same memory area). Therefore, each program group is coded relative to address 0000. Using the fill keyword (;) and the modifier (/), the programmer can now place the

program into any arbitrary area without disrupting the relationship of the instructions to one another. Instructions within such a group that are not to be modified must be preceded by an `X`.

On the other hand, the modification of an entire instruction group can be avoided by setting the modifier to zero. Thus, the availability of memory space is not restricted by the system.

[note: Bypassing the Loader: This note explains a simple and clean override for the relocatable loading process. If an engineer has already assembled or hand-coded a program using absolute disk/drum addresses (instead of coding it relative to 0000), they don't need a different loader utility. By simply explicitly initializing the modifier register to zero via the monitor console, the relocation math ($Address + Modifier$) becomes a transparent identity operation ($Address + 0$).True Memory Freedom: The phrase "the availability of memory space is not restricted by the system" is a proud architectural claim for the era. Many early vacuum-tube monitors forced rigid memory maps, locking away specific tracks or buffers exclusively for system use. J1-10.1, by design, lets the programmer treat almost the entire magnetic drum as a fluid canvas—allowing absolute code, relocatable blocks, and raw hex arrays to sit dynamically wherever the operator chooses to thread them.]

SUMMARY

Input Keyword Interpreted As Description ØTTSS Command – modifiable Instruction to be altered by the relocation modifier register. XØTTSS Command – not modifiable Absolute instruction; bypasses the modifier register. -ØTTSS Negative command – modifiable Command with negative sign bit or inverted logic flag; modifiable. -XØTTSS Negative command – not modifiable Command with negative sign bit or inverted logic flag; absolute. ;000TTSS Fill Keyword (Start Fill) Store the following words modifier excluded) in consecutive cells starting at TTSS. /000TTSS Modifier Set the modifier cell to TTSS. Add this modifier to all subsequent modifiable commands. .000TTSS Stop and Jump (Stop and Transfer) Halt (unless PS-32 is pressed) and jump to cell TTSS when a start signal occurs. +00ØTTSS Direct Command Permits the immediate execution of a command, where a second word may act as a data word. This second word must be in hexadecimal notation. ,00000NN Hexadecimal Words The next NN hexadecimal words are to be stored consecutively in the cells defined by the fill counter ($1 \leq NN \leq 63$). VNNCHHHH Hexadecimal Fill Store NN hexadecimal words starting at HHHH ($1 \leq NN \leq 63$). The keyword is in hexadecimal notation. For each group of words, a checksum is calculated, and, if PST is off, this checksum is compared with the checksum punched on the tape after the NN words. If PST is on, no checksum comparison takes place. A00000NN Alphanumeric Words The next NN words are to be stored in consecutive cells as alphanumeric words, each 5 six-bit characters long at $q = 29$ ($1 \leq NN \leq 63$). -000TTSS Inquiry Keyword (Frageschlüsselwort) Bring the word at TTSS into the accumulator. Upon pressing START, bring the word at TTSS+1 into the accumulator, and so on. Interrupt this mode of operation by executing a manual jump to 0000. S000TTOO Input Selection (Eingabeauswahl) Use the input unit whose identification number is TT for continuous input by the subroutine.

erminology Summary:

Ø stands for any arbitrary machine instruction/command.

TT are the decimal digits for the track address.

SS are the decimal digits for the sector address.

NN are the decimal digits indicating the quantity of words to be read in.

HHHH is an address expressed in hexadecimal notation.

[notes: Early Dynamic Linking and Relocation: This passage provides a phenomenal look at how the J1-10.1 monitor implements a crude form of relocatable object code assembly. By standardizing all independent subroutines to source-link relative to a baseline address of 0000, the monitor functions essentially as a dynamic loader. When the paper tape is read, the monitor intercepts the instructions on the fly, applies the relocation offset value currently held in the modifier register, and patches the addresses before laying them down on the drum.

The Absolute Override (`X`): The use of the textttX prefix is a brilliant mechanical solution for absolute addressing within a relocatable block. If a subroutine sitting at a variable location on the drum still needs to make a hard call to a fixed system resource (such as a hardware track or a specific core monitor vector), prefixing the instruction with `X` tells the `J1-10.1` parser to bypass the modifier addition loop for that specific line of code.

Instruction Decoding Internals: The remark that the monitor evaluates the sign bit and the three most significant bits to route control is a valuable architectural detail. In the LGP machine word layout, this specific bit slice contains the raw operational flags that distinguish an executable instruction or monitor macro from a raw data pattern, enabling the fast software branch table inside J1-10.1.]

Translator notes

Rather than a loader J1-10.1 is more of a programmer's interactive monitor.

Comparing this to J1-10.0 (the 4-track version), J1-10.1 uses the Flexowriter stop code (') to terminate a command. J1-10.0 does not.

This means J1-10.1 relies heavily on reactive interactive execution steps via the Flexowriter keyboard, requiring you to physically hit the START COMP lever or console button to advance past the initial track selection bracket.

9 Data Input

9.1 Data Input 1

9.2 Data Input 2

9.3 Data Input 3

9.4 Data Input 4

9.5 Data Input 5

10 Data Output

10.1 Data Output 1

10.2 Data Output 2

10.3 Data Output 3

10.4 Data Output 4

10.5 Data Output 5

10.6 Data Output 6

11 Hexadecimal Output

[Translator's note: there is no Hexadecimal Output 1 program in this document.]

11.1 Hexadecimal Output 2

11.2 Hexadecimal Output 3

11.3 Hexadecimal Output 4

12 Alphanumeric Output

12.1 Alphanumeric Output 1

12.2 Alphanumeric Output 2

13 Hexadecimal Input

13.1 Hexadecimal Input 1

14 Specialized Input/Output

14.1 Angle or Direction I/O

15 Tracing

15.1 Trace 1

16 Memory Dumps

16.1 Memory Dump 1

16.2 Memory Dump 2

17 Sorting

17.1 Sort 1

17.2 Sort 2

17.3 Sort 3

17.4 Sort 4

18 Conversion to Binary

18.1 Binary Conversion of Integers 1

19 Backup Utilities

19.1 Backup 1

19.2 Tape Duplicator 1

20 Binary Searching

20.1 Table Search 1 (Binary Search)

21 Programming Utilities

21.1 Loop Driver Utility

22 Device Support

22.1 High-speed Punch Output

23 Bibliography

References

[1] J. von Neumann, “First Draft of a Report on the EDVAC,” Moore School of Electrical Engineering, University of Pennsylvania, Philadelphia, PA, Tech. Rep., Jun. 1945.

[2] A. W. Burks, H. H. Goldstine, and J. von Neumann, “Preliminary discussion of the logical design of an electronic computing instrument,” Institute for Advanced Study, Princeton, NJ, Tech. Rep., Jun. 1946.

[3] International Business Machines Corporation, *Principles of Operation, Electronic Data Processing Machines Type 701*, New York, NY, 1953.

[4] Digital Computer Laboratory, *Electronic Digital Computer: Internal Organization, Programming, and Coding (ILLIAC I)*, University of Illinois, Urbana, IL, 1954.

[5] Elliott Brothers (London) Ltd., *The Elliott 405 Electronic Data Processing System: Reference Manual*, London, UK, 1956.

[6] Royal McBee Computer Corporation, *LGP-30 Programming Manual*, Port Chester, NY, 1957.

[7] General Precision Electronic Computer Company, *LGP-21 Maintenance and Training Manual*, Burbank, CA, 1963

[8] Eurocomp GmbH, *LGP-21 Unterprogramm-Handbuch*, Minden, Westphalia, 1963

[9] General Precision Electronic Computer Company, Burbank, CA, 1963

[10] General Precision Electronic Computer Company, *RPC-4000 Programmers Reference Manual*, Burbank, CA, 1960.

[11] E. Nigh, “The Story of Mel,” first posted to Usenet newsgroup `net.followup`, May 1983. [Online; accessed June 2026]. Available: `https://users.cs.utah.edu/~elb/folklore/mel.html`